Practical No. 1
Data Pre-processing and Exploration

(A). Load a CSV dataset. Handle missing values, inconsistent formatting, and outliers. Here are the general steps for cleaning a dataset:

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
# Load the dataset from a CSV file
file_path = 'data.csv'  # Replace with your file path
df = pd.read_csv(file_path)
# Display the first few rows
print(df.head())
# Handle Missing Values
print("Missing values before cleaning:")
print(df.isnull().sum())
# Fill missing values with mean for numerical columns and mode for categorical columns
df['numerical_column'] = df['numerical_column'].fillna(df['numerical_column'].mean())
df['category_column'] = df['category_column'].fillna(df['category_column'].mode()[0])
# Handle Inconsistent Formatting (Example: Strip spaces and convert to lowercase)
df.columns = df.columns.str.strip()  # Strip spaces from column names
df['category_column'] = df['category_column'].str.lower()  # Convert text to lowercase
# Handle Outliers (Using IQR)
Q1 = df['numerical_column'].quantile(0.25)
Q3 = df['numerical_column'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df_cleaned = df[(df['numerical_column'] >= lower_bound) & (df['numerical_column'] <=
upper_bound)]
# Display cleaned data
print("Cleaned data:")
print(df_cleaned.head())
# Visualize with Boxplot to check for outliers
sns.boxplot(x=df_cleaned['numerical_column'])
plt.title('Boxplot of Numerical Column After Cleaning')
plt.show()

# Visualize the distribution with a histogram
df_cleaned['numerical_column'].hist(bins=30)
plt.title('Histogram of Numerical Column After Cleaning')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()
```

Output:

```
<bound method NDFrame.head of    ID     Name   Age    Income  Join_Date
0   1     John  25.0   50000.0 2020-01-10
1   2     Mary  35.0   70000.0 2021-03-15
2   3      Bob  45.0  120000.0 2020-02-20
3   4    Alice  35.0   85000.0 2020-05-30
4   5  Charlie  28.0   85000.0 2019-09-10
5   6  Unknown  38.0   95000.0 2021-12-25
6   7    David  50.0  264000.0 2020-10-01
7   8     Emma  30.0   45000.0 2018-07-18
8   9    Frank  22.0   52000.0 2022-04-05
9  10    Grace  70.0  220000.0 2020-01-15>
>>>
```

(B). Load a dataset, calculate descriptive summary statistics, create visualizations using different graphs, and identify potential features and target variables
Note: Explore Univariate and Bivariate graphs (Matplotlib) and Seaborn for visualization.

Code:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Load the Iris dataset
df = sns.load_dataset("iris")

# 1. Descriptive Statistics
print(df.describe())

# 2. Univariate Visualizations
# a. Histogram for 'sepal_length'
plt.figure(figsize=(8, 6))
sns.histplot(df['sepal_length'], kde=True, bins=20, color='blue')
plt.title('Distribution of Sepal Length')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Frequency')
plt.show()

# b. Boxplot for 'petal_length'
plt.figure(figsize=(8, 6))
sns.boxplot(x=df['petal_length'], color='green')
plt.title('Boxplot of Petal Length')
plt.xlabel('Petal Length (cm)')
plt.show()

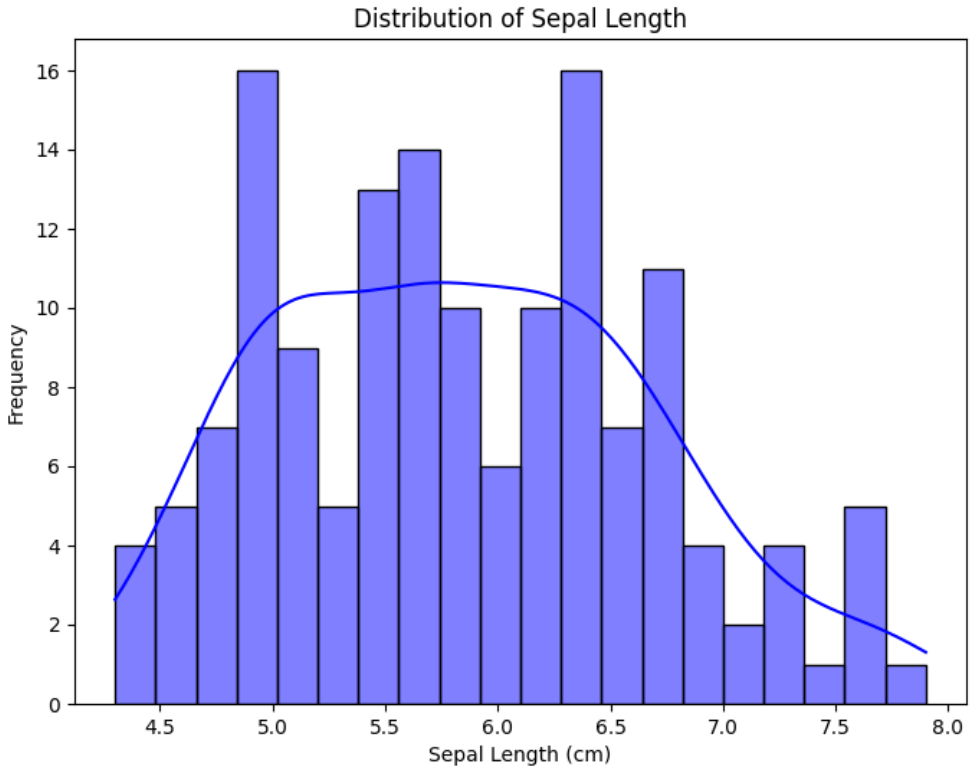
# c. Density plot for 'sepal_width'
plt.figure(figsize=(8, 6))
sns.kdeplot(df['sepal_width'], shade=True, color='red')
plt.title('Density Plot of Sepal Width')
plt.xlabel('Sepal Width (cm)')
plt.ylabel('Density')
plt.show()

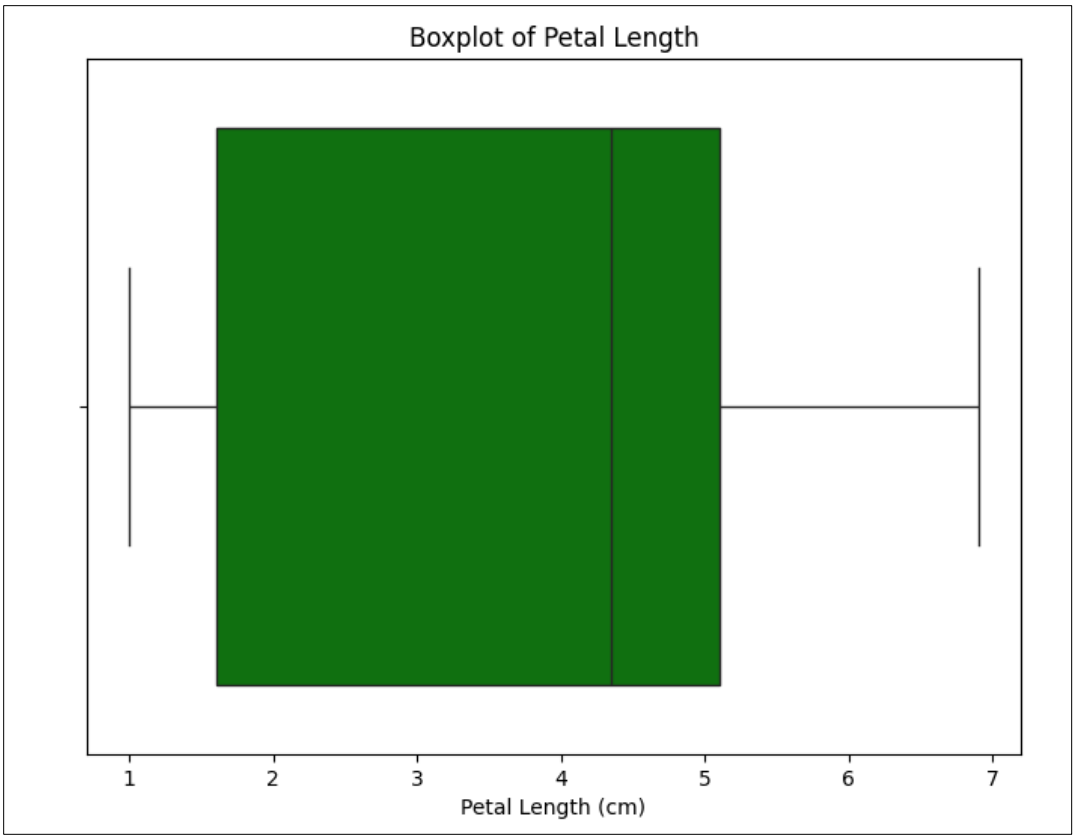
# 3. Bivariate Visualizations
# a. Scatter plot for Sepal Length vs Sepal Width
plt.figure(figsize=(8, 6))
sns.scatterplot(x='sepal_length', y='sepal_width', data=df, hue='species')
plt.title('Sepal Length vs Sepal Width')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.show()
# b. Pairplot of all numerical variables
sns.pairplot(df, hue='species')
plt.show()
# c. Correlation Heatmap
```

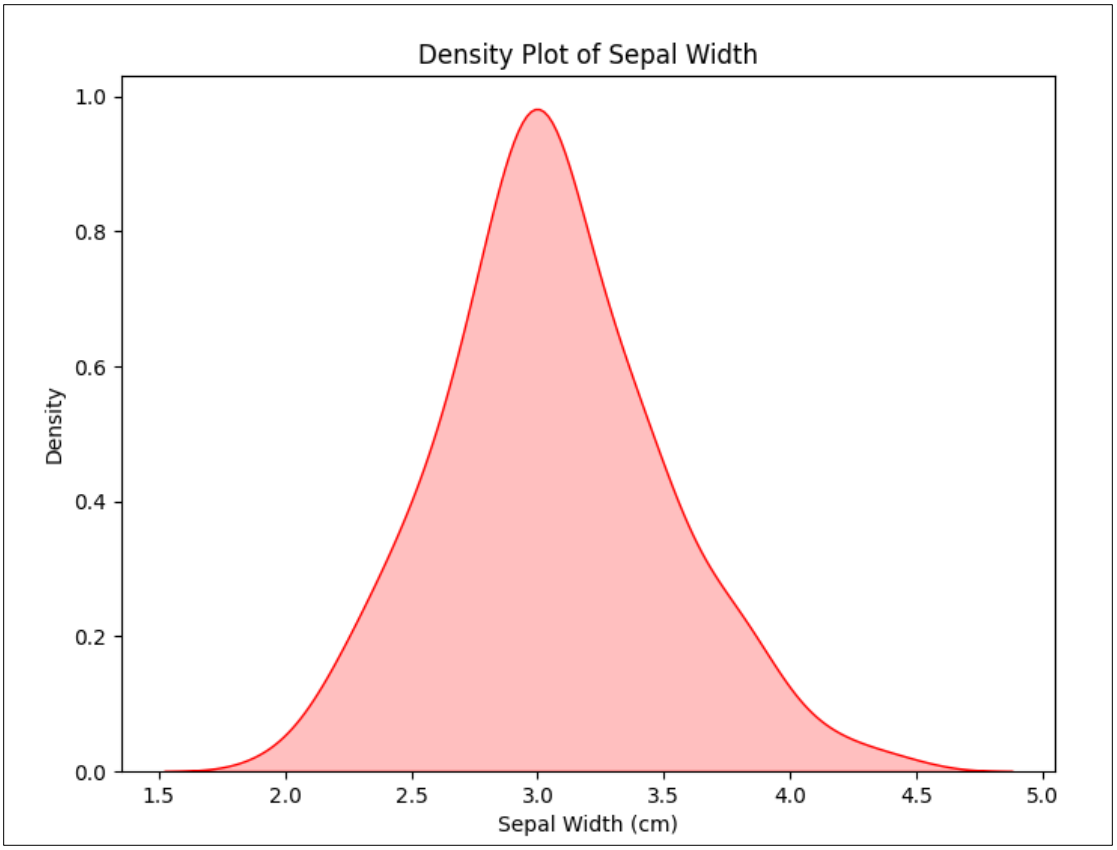
```
corr_matrix = df.corr()
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title('Correlation Heatmap of Numerical Features')
plt.show()
```

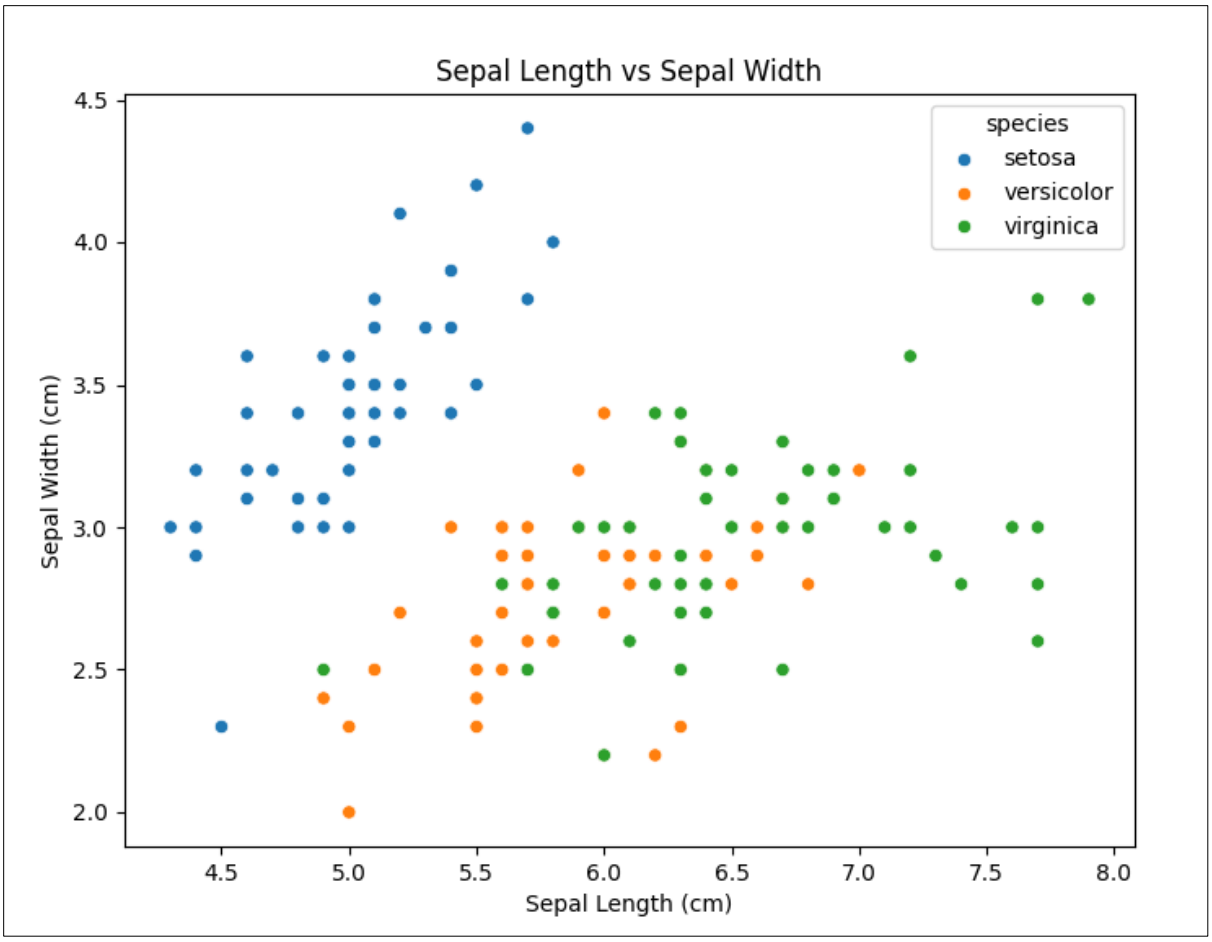
Output:

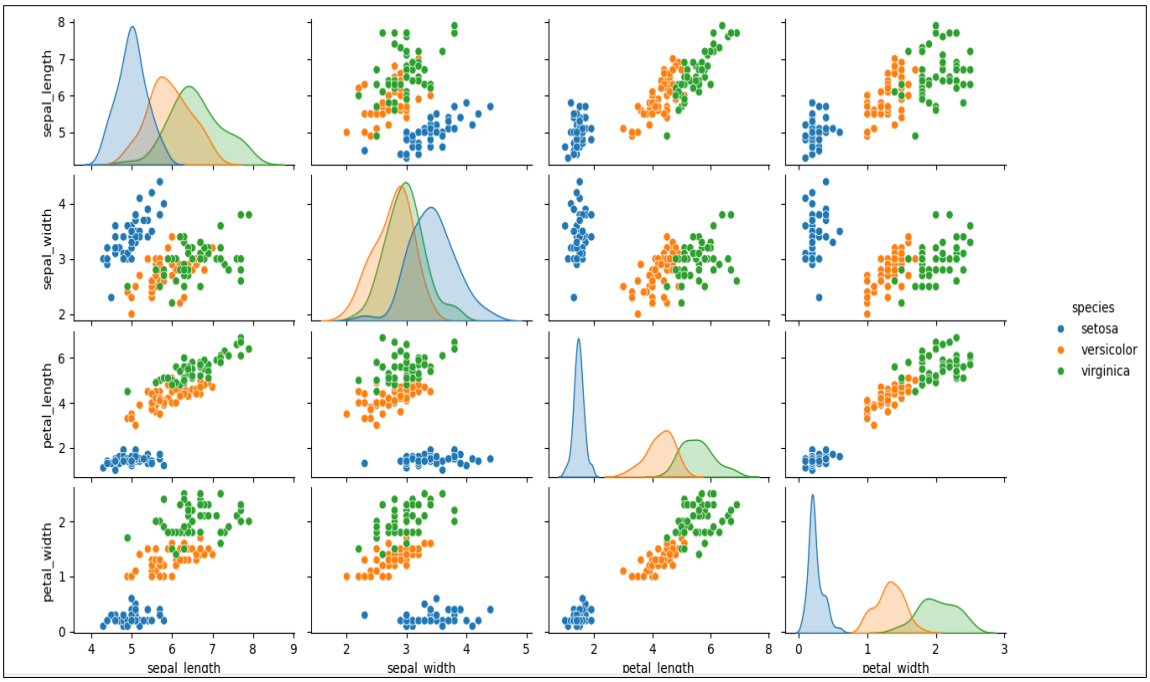
	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000




Boxplot of Petal Length
1
2
3
4
5
6
7
Petal Length (cm)


Density Plot of Sepal Width
1.0
0.8
0.6
0.4
0.2
0.0
Density
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0
Sepal Width (cm)


Sepal Length vs Sepal Width
Sepal Width (cm)
Sepal Length (cm)
species
setosa
versicolor
virginica
4.5
4.0
3.5
3.0
2.5
2.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0


sepal_length
sepal_width
petal_length
petal_width
species
setosa
versicolor
virginica

(C). Create or Explore datasets to use all pre-processing routines like label encoding, scaling, and binarization.

Solution:

```
# Step 1: Import necessary libraries
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler, Binarizer

# Step 2: Create a synthetic dataset
data = {
    "Numerical_Column": [10, 20, 30, 40, 50],
    "Categorical_Column": ["Low", "Medium", "High", "Medium", "Low"],
    "Binary_Column": [1, 0, 1, 0, 1]
}

# Convert to a DataFrame
df = pd.DataFrame(data)

# Step 3: Handle missing values (if any)
# Here we simulate missing values and fill them
df.loc[1, "Numerical_Column"] = np.nan
df["Numerical_Column"].fillna(df["Numerical_Column"].mean(), inplace=True)

# Step 4: Apply Label Encoding to the categorical column
label_encoder = LabelEncoder()
df["Categorical_Column_Encoded"] = label_encoder.fit_transform(df["Categorical_Column"])

# Step 5: Scale numerical columns
scaler = StandardScaler()
df["Numerical_Scaled"] = scaler.fit_transform(df[["Numerical_Column"]])

# Min-Max Scaling for comparison
min_max_scaler = MinMaxScaler()
df["Numerical_MinMax_Scaled"] = min_max_scaler.fit_transform(df[["Numerical_Column"]])

# Step 6: Apply Binarization to numerical columns
binarizer = Binarizer(threshold=25)
df["Numerical_Binarized"] = binarizer.fit_transform(df[["Numerical_Column"]])

# Print the resulting DataFrame
print(df)
```

Output:

```
   Numerical_Column Categorical_Column  Binary_Column  Categorical_Column_Encoded  Numerical_Scaled  Numerical_MinMax_Scaled  Numerical_Binarized
0              10.0                Low              1                           1         -1.700840                   0.0000                  0.0
1              32.5             Medium              0                           2          0.000000                   0.5625                  1.0
2              30.0               High              1                           0         -0.188982                   0.5000                  1.0
3              40.0             Medium              0                           2          0.566947                   0.7500                  1.0
4              50.0                Low              1                           1          1.322876                   1.0000                  1.0
```

Practical No. 2
Testing Hypothesis

(A). Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a. CSV file and generate the final specific hypothesis. (Create your dataset)

Code:

```
import pandas as pd
# Step 1: Create a simple dataset and save it to a CSV file
data = {
    "Sky": ["Sunny", "Sunny", "Rainy", "Sunny", "Sunny"],
    "AirTemp": ["Warm", "Warm", "Cold", "Warm", "Warm"],
    "Humidity": ["Normal", "High", "High", "High", "Normal"],
    "Wind": ["Strong", "Strong", "Strong", "Strong", "Strong"],
    "Water": ["Warm", "Warm", "Warm", "Cool", "Warm"],
    "Forecast": ["Same", "Same", "Change", "Same", "Same"],
    "EnjoySport": ["Yes", "Yes", "No", "Yes", "Yes"]
}

# Save dataset to CSV for demonstration purposes
df = pd.DataFrame(data)
df.to_csv("training_data.csv", index=False)

# Step 2: Load the training data from CSV
training_data = pd.read_csv("training_data.csv")

# Step 3: Implement FIND-S algorithm
def find_s_algorithm(data):
    # Extract features and target
    features = data.iloc[:, :-1].values  # All columns except the last
    target = data.iloc[:, -1].values     # The last column (target)
    # Initialize the most specific hypothesis
    hypothesis = ["ϕ"] * features.shape[1]  # Start with the most specific hypothesis
    # Update hypothesis for positive examples
    for i, row in enumerate(features):
        if target[i] == "Yes":  # Consider only positive examples
            for j in range(len(hypothesis)):
                if hypothesis[j] == "ϕ":  # Update if still specific
                    hypothesis[j] = row[j]
                elif hypothesis[j] != row[j]:  # Generalize if mismatch
                    hypothesis[j] = "?"
    return hypothesis
# Step 4: Run the FIND-S algorithm
final_hypothesis = find_s_algorithm(training_data)
# Step 5: Display the results
print("Final Specific Hypothesis:", final_hypothesis)
```

Output:

```
Final Specific Hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', 'Same']
>>>
```

Practical No. 3
Linear Models

(A) Simple Linear Regression
Fit a linear regression model on a dataset. Interpret coefficients, make predictions, and evaluate performance using metrics like R-squared and MSE

Code:

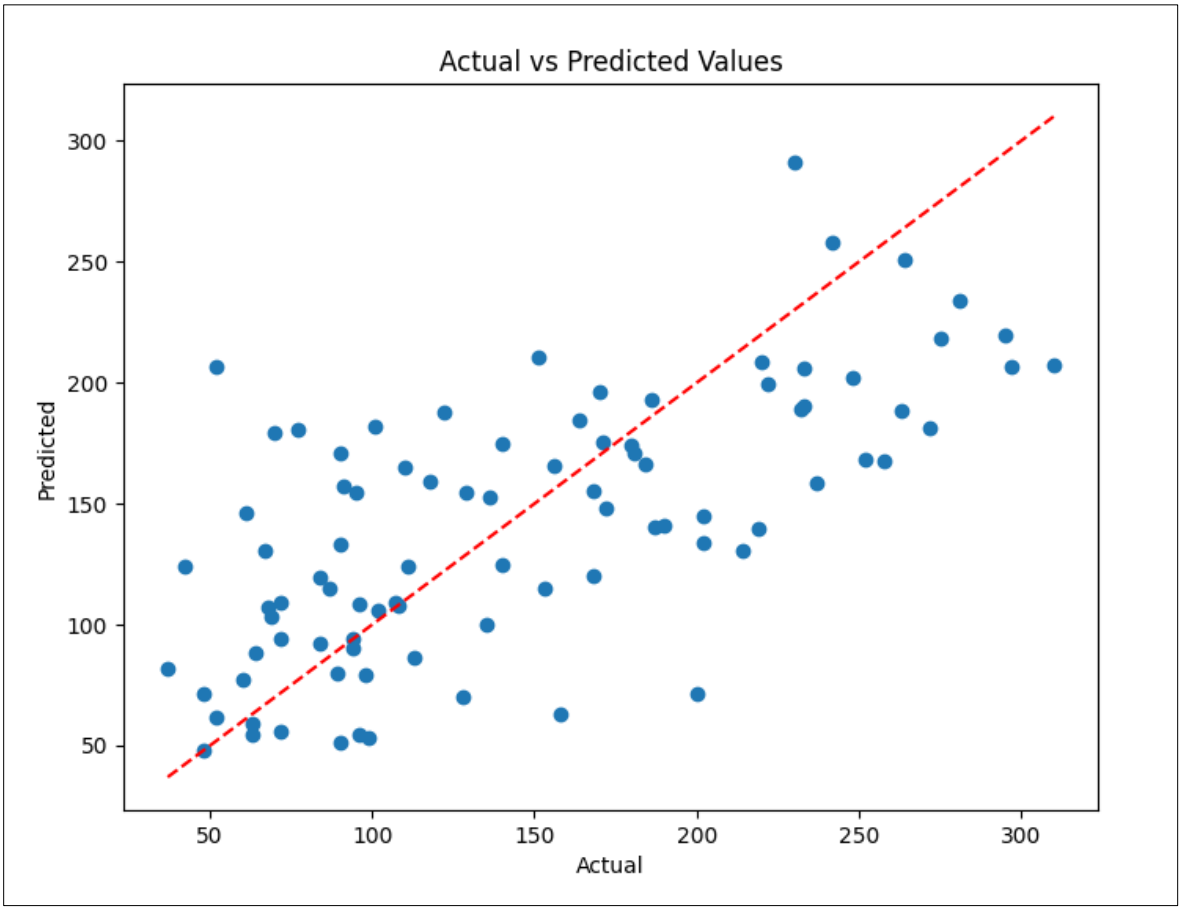
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import load_diabetes

# Load the diabetes dataset
data = load_diabetes()
# Convert the dataset to a pandas DataFrame
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
# Show the first few rows of the dataframe
print(df.head())
# Split the data into features (X) and target (y)
X = df.drop('target', axis=1)
y = df['target']
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Create a linear regression model
model = LinearRegression()
# Train the model
model.fit(X_train, y_train)
# Make predictions
y_pred = model.predict(X_test)
# Calculate the mean squared error and R2 score
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Mean Squared Error: {mse}")
print(f"R2 Score: {r2}")
# Visualize the predicted vs. actual values
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred)
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', linestyle='--')
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Actual vs Predicted Values")
plt.show()
```

Output:

```
        age       sex       bmi        bp  ...        s4        s5        s6  target
0  0.038076  0.050680  0.061696  0.021872  ... -0.002592  0.019907 -0.017646   151.0
1 -0.001882 -0.044642 -0.051474 -0.026328  ... -0.039493 -0.068332 -0.092204    75.0
2  0.085299  0.050680  0.044451 -0.005670  ... -0.002592  0.002861 -0.025930   141.0
3 -0.089063 -0.044642 -0.011595 -0.036656  ...  0.034309  0.022688 -0.009362   206.0
4  0.005383 -0.044642 -0.036385  0.021872  ... -0.002592 -0.031988 -0.046641   135.0

[5 rows x 11 columns]
Mean Squared Error: 2900.19362849348
R2 Score: 0.4526027629719197
```



(B). Multiple Linear Regression
Extend linear regression to multiple features. Handle feature selection and potential multicollinearity.

Code:

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant
from sklearn.datasets import load_diabetes

# Load the Diabetes dataset
diabetes = load_diabetes()

# Convert it into a pandas DataFrame
df = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)

# Add the target variable (diabetes progression) to the dataframe
df['PROGRESSION'] = diabetes.target

# Display the first few rows of the dataset
print("Dataset Overview:")
print(df.head(), "\n")

# Check for multicollinearity using the correlation matrix
corr_matrix = df.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title("Correlation Matrix")
plt.show()

# Check Variance Inflation Factor (VIF) to detect multicollinearity
X = df.drop(columns=['PROGRESSION'])
X_with_const = add_constant(X)  # Add constant to the features
vif_data = pd.DataFrame()
vif_data["Feature"] = X_with_const.columns
vif_data["VIF"] = [variance_inflation_factor(X_with_const.values, i) for i in
range(X_with_const.shape[1])]

print("\nVariance Inflation Factor (VIF):")
print(vif_data, "\n")
```

```
# Remove features with high VIF (>10), or use Lasso Regression to handle feature selection
X_selected = X.drop(columns=['s5', 's4'])  # Drop highly correlated features, based on VIF analysis

# Split the dataset into features (X) and target (y)
X = X_selected
y = df['PROGRESSION']

# Standardize the features (important for regularization)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split the data into training and testing sets (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

# Initialize and train the Multiple Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Get the model's coefficients and intercept
intercept = model.intercept_
coefficients = model.coef_

print(f"\nIntercept: {intercept}")
print(f"Coefficients: {coefficients}\n")

# Make predictions using the test set
y_pred = model.predict(X_test)

# Compare actual vs predicted values
comparison_df = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
print("Actual vs Predicted Progression (for the first 5 records):")
print(comparison_df.head(), "\n")

# Calculate R-squared (R²) and Mean Squared Error (MSE)
r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)

print(f"R-squared: {r2}")
print(f"Mean Squared Error (MSE): {mse}\n")

# Visualize the regression line (if it were 2D) or scatter plot of actual vs predicted
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', linestyle='--')
plt.title('Actual vs Predicted Progression')
plt.xlabel('Actual Progression')
```

```
plt.ylabel('Predicted Progression')
plt.show()

# Regularization using Lasso (L1) Regression to perform feature selection
lasso = Lasso(alpha=0.1)  # Regularization strength (alpha)
lasso.fit(X_train, y_train)

# Display Lasso coefficients (some of them might be zero, meaning the feature is not important)
lasso_coefficients = lasso.coef_
print(f"Lasso Coefficients: {lasso_coefficients}\n")

# Make predictions using Lasso model
y_pred_lasso = lasso.predict(X_test)

# Calculate R-squared and MSE for Lasso model
r2_lasso = r2_score(y_test, y_pred_lasso)
mse_lasso = mean_squared_error(y_test, y_pred_lasso)

print(f"Lasso R-squared: {r2_lasso}")
print(f"Lasso Mean Squared Error (MSE): {mse_lasso}\n")

# Plot Lasso regression predictions
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred_lasso, color='green')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', linestyle='--')
plt.title('Actual vs Predicted Progression (Lasso Regression)')
plt.xlabel('Actual Progression')
plt.ylabel('Predicted Progression (Lasso)')
plt.show()
```

Output:

```
Dataset Overview:
        age       sex       bmi  ...        s5        s6  PROGRESSION
0  0.038076  0.050680  0.061696  ...  0.019907 -0.017646        151.0
1 -0.001882 -0.044642 -0.051474  ... -0.068332 -0.092204         75.0
2  0.085299  0.050680  0.044451  ...  0.002861 -0.025930        141.0
3 -0.089063 -0.044642 -0.011595  ...  0.022688 -0.009362        206.0
4  0.005383 -0.044642 -0.036385  ... -0.031988 -0.046641        135.0

[5 rows x 11 columns]


Variance Inflation Factor (VIF):
   Feature        VIF
0    const   1.000000
1      age   1.217307
2      sex   1.278071
3      bmi   1.509437
4       bp   1.459428
5       s1  59.202510
6       s2  39.193370
7       s3  15.402156
8       s4   8.890986
9       s5  10.075967
10      s6   1.484623


Intercept: 151.28459747294644
Coefficients: [  2.5697767  -11.43487549  28.38406996  16.69107341  35.30502059
 -35.56465308 -29.67965409   4.0550404 ]
```

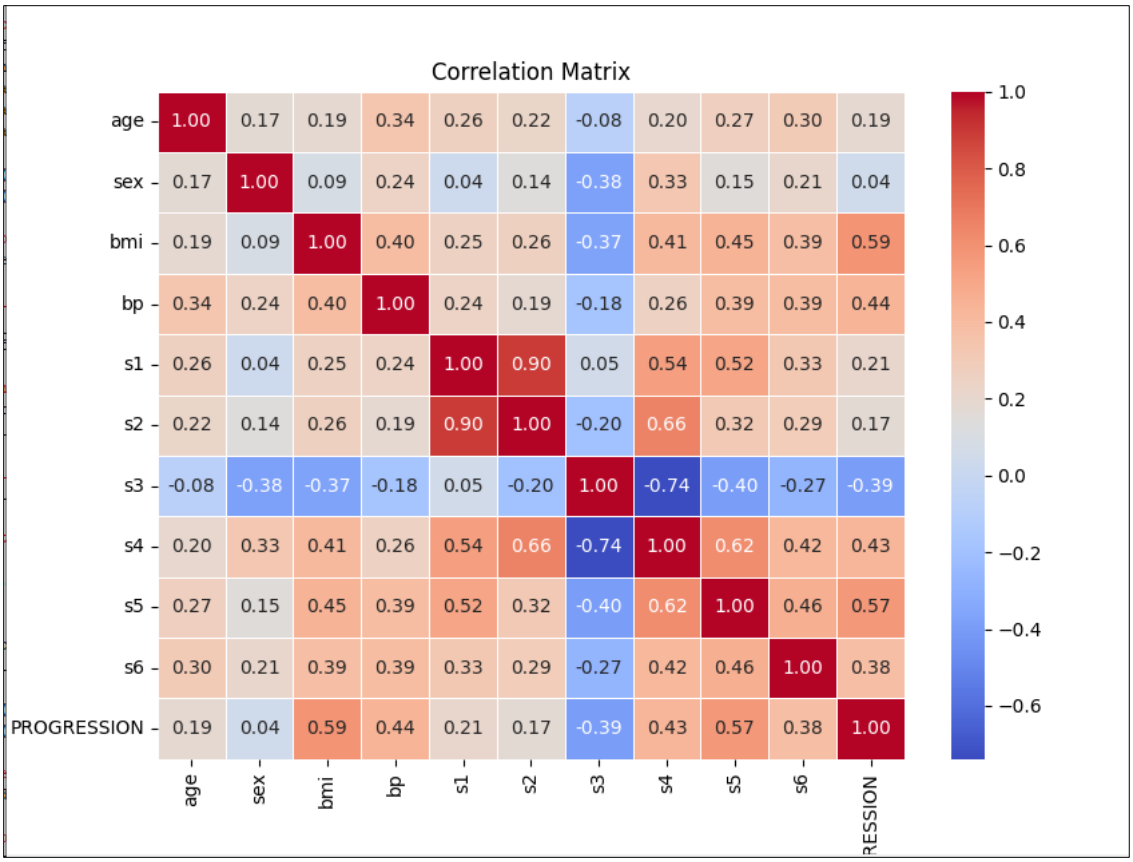
```
Actual vs Predicted Progression (for the first 5 records):
     Actual   Predicted
287   219.0  141.914888
211    70.0  188.301763
72    202.0  139.232853
321   230.0  292.377931
73    111.0  118.942975

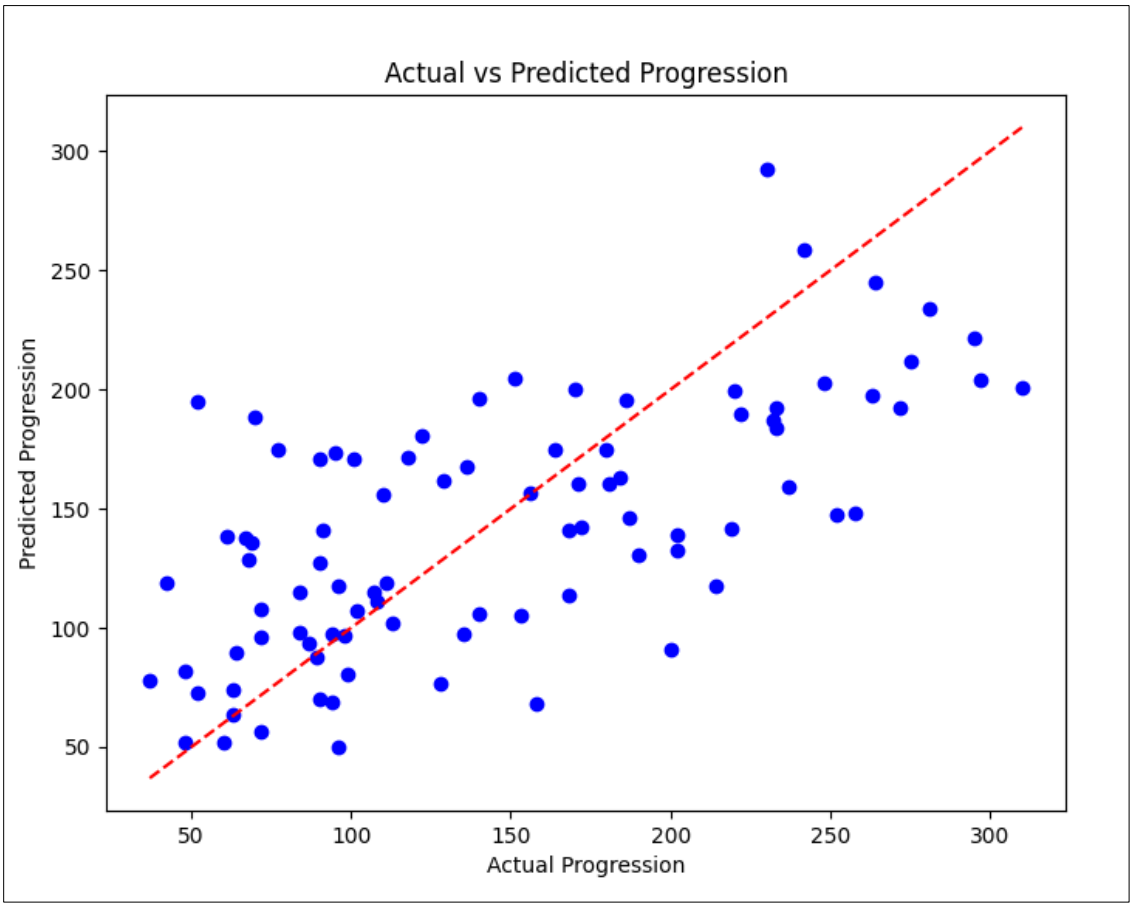
R-squared: 0.4318382847188821
Mean Squared Error (MSE): 3010.2069852568466

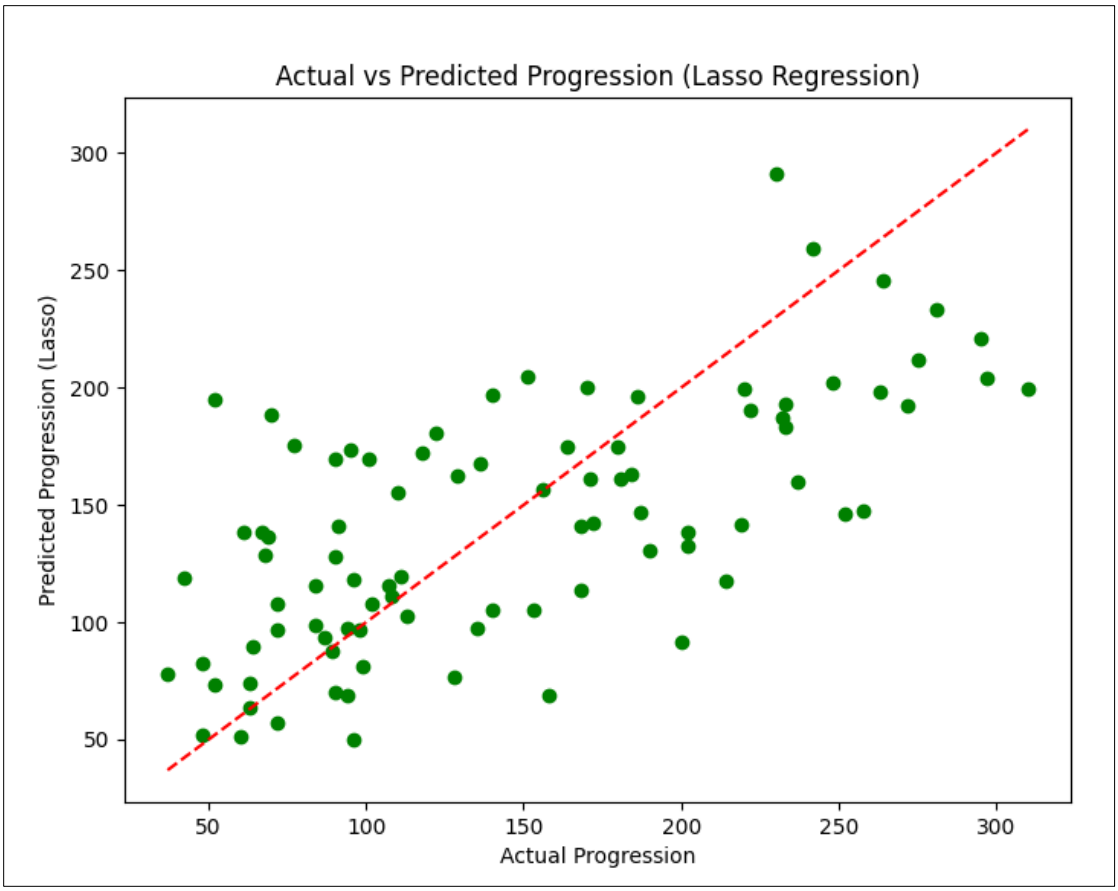
Lasso Coefficients: [  2.4993986  -11.29047156  28.45971324  16.71676986  33.805
95539
 -34.0654115  -29.10515762   4.13179395]

Lasso R-squared: 0.4302105909413533
Lasso Mean Squared Error (MSE): 3018.8307538903105

>>>
```


Correlation Matrix
age 1.00 0.17 0.19 0.34 0.26 0.22 -0.08 0.20 0.27 0.30 0.19
sex 0.17 1.00 0.09 0.24 0.04 0.14 -0.38 0.33 0.15 0.21 0.04
bmi 0.19 0.09 1.00 0.40 0.25 0.26 -0.37 0.41 0.45 0.39 0.59
bp 0.34 0.24 0.40 1.00 0.24 0.19 -0.18 0.26 0.39 0.39 0.44
s1 0.26 0.04 0.25 0.24 1.00 0.90 0.05 0.54 0.52 0.33 0.21
s2 0.22 0.14 0.26 0.19 0.90 1.00 -0.20 0.66 0.32 0.29 0.17
s3 -0.08 -0.38 -0.37 -0.18 0.05 -0.20 1.00 -0.74 -0.40 -0.27 -0.39
s4 0.20 0.33 0.41 0.26 0.54 0.66 -0.74 1.00 0.62 0.42 0.43
s5 0.27 0.15 0.45 0.39 0.52 0.32 -0.40 0.62 1.00 0.46 0.57
s6 0.30 0.21 0.39 0.39 0.33 0.29 -0.27 0.42 0.46 1.00 0.38
PROGRESSION 0.19 0.04 0.59 0.44 0.21 0.17 -0.39 0.43 0.57 0.38 1.00
age sex bmi bp s1 s2 s3 s4 s5 s6
1.0 0.8 0.6 0.4 0.2 0.0 −0.2 −0.4 −0.6


Actual vs Predicted Progression
Predicted Progression
Actual Progression
50 100 150 200 250 300


Actual vs Predicted Progression (Lasso Regression)
Predicted Progression (Lasso)
Actual Progression
50
100
150
200
250
300

(C) . Regularized Linear Models (Ridge, Lasso, Elastic Net)
Implement regression variants like LASSO and Ridge on any generated dataset.

Code:

```
# Importing necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.metrics import mean_squared_error, r2_score
# Step 1: Generate a synthetic dataset
X, y = make_regression(n_samples=100, n_features=10, noise=0.1, random_state=42)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 2: Apply Ridge, Lasso, and ElasticNet regression models
ridge = Ridge(alpha=1.0)
lasso = Lasso(alpha=0.1)
elastic_net = ElasticNet(alpha=0.1,l1_ratio=0.5)  # l1_ratio = 0.5 is a good start for mixing L1 and L2

# Train the models
ridge.fit(X_train, y_train)
lasso.fit(X_train, y_train)
elastic_net.fit(X_train, y_train)
# Step 3: Predictions
y_pred_ridge = ridge.predict(X_test)
y_pred_lasso = lasso.predict(X_test)
y_pred_elastic_net = elastic_net.predict(X_test)
# Step 4: Evaluate the models using MSE and R² score
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
mse_lasso = mean_squared_error(y_test, y_pred_lasso)
mse_elastic_net = mean_squared_error(y_test, y_pred_elastic_net)
r2_ridge = r2_score(y_test, y_pred_ridge)
r2_lasso = r2_score(y_test, y_pred_lasso)
r2_elastic_net = r2_score(y_test, y_pred_elastic_net)

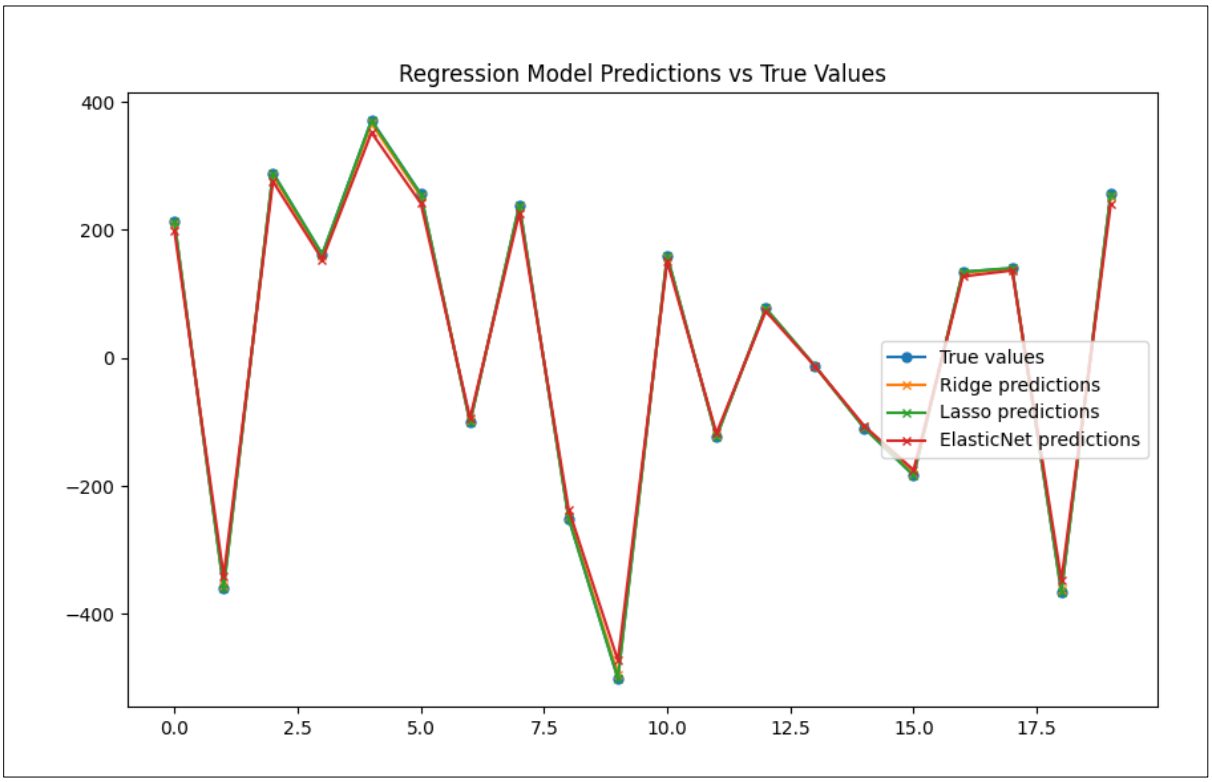
# Print the results
print(f"Ridge Regression MSE: {mse_ridge:.4f}, R²: {r2_ridge:.4f}")
print(f"Lasso Regression MSE: {mse_lasso:.4f}, R²: {r2_lasso:.4f}")
print(f"ElasticNet Regression MSE: {mse_elastic_net:.4f}, R²: {r2_elastic_net:.4f}")

# Step 5: Plot the results (optional, just for visualization)
plt.figure(figsize=(10, 6))
plt.plot(y_test, label='True values', linestyle='-', marker='o', markersize=5)
plt.plot(y_pred_ridge, label='Ridge predictions', linestyle='-', marker='x', markersize=5)
plt.plot(y_pred_lasso, label='Lasso predictions', linestyle='-', marker='x', markersize=5)
plt.plot(y_pred_elastic_net, label='ElasticNet predictions', linestyle='-', marker='x', markersize=5)
```

```
plt.legend()
plt.title('Regression Model Predictions vs True Values')
plt.show()
```

Output:

```
Ridge Regression MSE: 11.8446, R²: 0.9998
Lasso Regression MSE: 0.1824, R²: 1.0000
ElasticNet Regression MSE: 176.0283, R²: 0.9971
```



Practical No. 4
Logistic Regression

(A). Perform binary classification using logistic regression. Calculate accuracy, precision, recall, and understand the ROC curve.

Note:
pip install --upgrade scikit-learn

Step 1: Import Required Libraries
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_curve, auc
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Load and Prepare the Dataset
For demonstration purposes, let's use a sample dataset like the famous Iris dataset or any binary classification dataset.
```
# Sample dataset: load it using scikit-learn
from sklearn.datasets import load_iris
iris = load_iris()

# We will use only two classes (binary classification) from the Iris dataset
X = iris.data[iris.target != 2]  # Features
y = iris.target[iris.target != 2]  # Target variable
```

Step 3: Split the Data into Training and Testing Sets
```
# Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Step 4: Train the Logistic Regression Model
```
# Initialize the Logistic Regression model
model = LogisticRegression()
# Train the model
model.fit(X_train, y_train)
```

Step 5: Make Predictions
```
# Make predictions on the test set
y_pred = model.predict(X_test)

# Predict probabilities for ROC curve (output probabilities for class 1)
y_prob = model.predict_proba(X_test)[:, 1]
```

Step 6: Evaluate the Model
We will now compute the accuracy, precision, recall, and plot the ROC curve.

1. **Accuracy**
   ```
   accuracy = accuracy_score(y_test, y_pred)
   print(f'Accuracy: {accuracy:.4f}')
   ```
2. **Precision**
   ```
   precision = precision_score(y_test, y_pred)
   ```

```
print(f'Precision: {precision:.4f}')
```

3. **Recall**

```
recall = recall_score(y_test, y_pred)
print(f'Recall: {recall:.4f}')
```

4. **ROC Curve**

To understand the ROC curve, we need to plot it using roc_curve and calculate the AUC (Area Under the Curve).

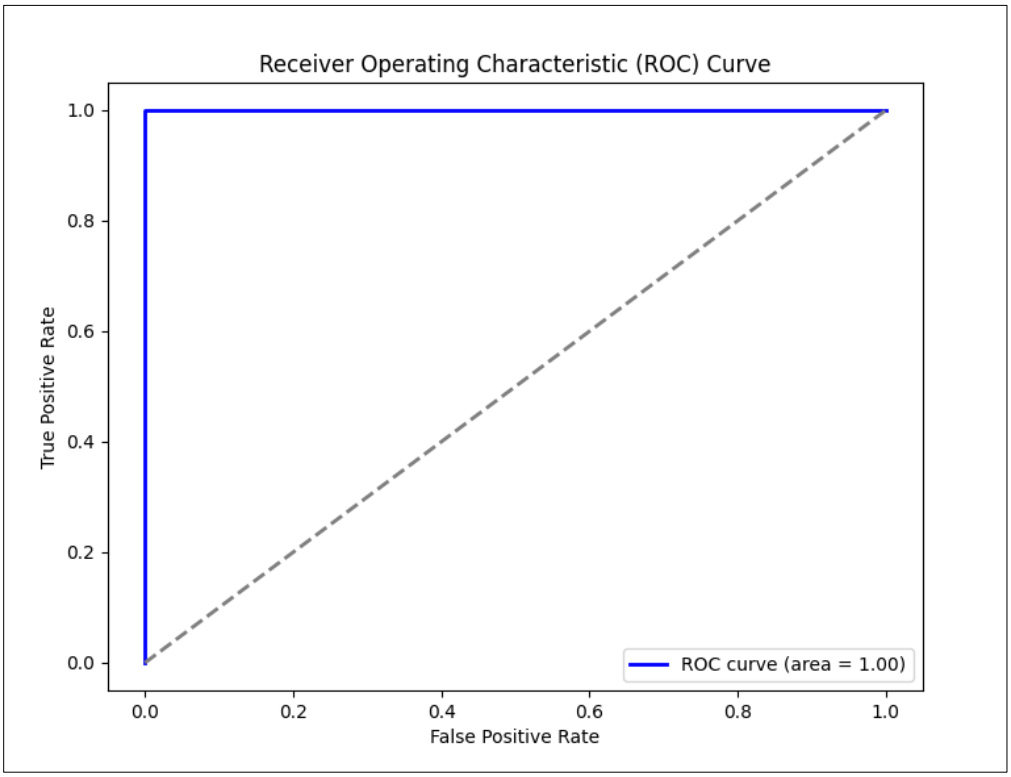
```
# Compute ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_prob)

# Compute the Area Under the Curve (AUC)
roc_auc = auc(fpr, tpr)

# Plot ROC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='blue', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='gray', lw=2, linestyle='--')  # Diagonal line (no skill)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.show()
```

Output:

```
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
>>>
```



(B). Implement and demonstrate k-nearest Neighbor algorithm. Read the training data from a .CSV file and build the model to classify a test sample. Print both correct and wrong predictions.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Step 1: Read training data from a CSV file
# Let's assume the CSV file has feature columns and the target column is named 'class'.
data = pd.read_csv('data.csv')  # Update with your actual file path

# Split the data into features (X) and target labels (y)
X = data.drop(columns=['class'])  # All columns except 'class'
y = data['class']  # The target column

# Step 2: Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 3: Initialize the k-NN classifier and train the model
k = 3  # You can change k to any number for the number of neighbors
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train, y_train)

# Step 4: Predict the labels for the test set
y_pred = knn.predict(X_test)

# Step 5: Print correct and wrong predictions with sample data
for i in range(len(X_test)):
    sample = X_test.iloc[i]
    true_label = y_test.iloc[i]
    predicted_label = y_pred[i]

    if true_label == predicted_label:
        print(f"Correct: Sample {i+1} -> Features: {sample.values} | True label: {true_label} | Predicted:
{predicted_label}")
    else:
        print(f"Wrong: Sample {i+1} -> Features: {sample.values} | True label: {true_label} | Predicted:
{predicted_label}")

# Step 6: Evaluate the accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"\nModel Accuracy: {accuracy * 100:.2f}%")
```

Output:

```
Wrong: Sample 1 -> Features: [2.3 1.2 4.5] | True label: 1 | Predicted: 0

Model Accuracy: 0.00%
>>>
```

(C). Build a decision tree classifier or regressor. Control hyperparameters like tree depth to avoid overfitting. Visualize the tree.

Code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, plot_tree
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# Step 1: Load a Sample Dataset
# For example, we use the iris dataset available in scikit-learn
iris = load_iris()
X = iris.data
y = iris.target

# Step 2: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

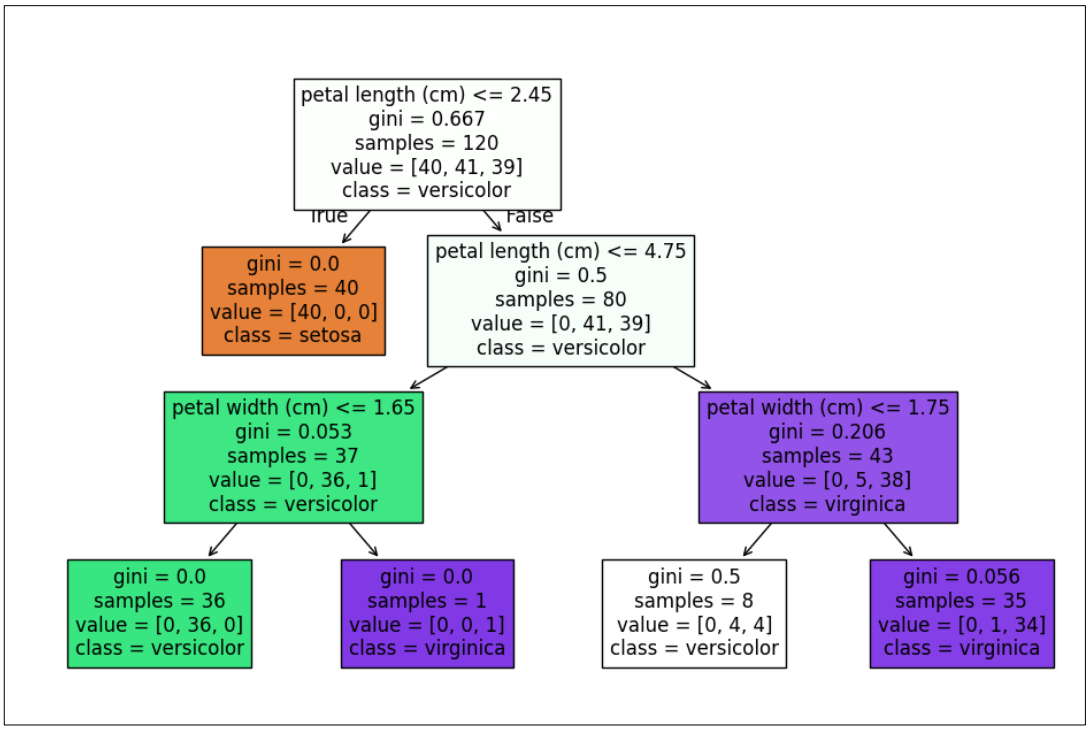
# Step 3: Create and Train a Decision Tree Classifier (Control max depth to avoid overfitting)
clf = DecisionTreeClassifier(max_depth=3, random_state=42)  # max_depth controls overfitting
clf.fit(X_train, y_train)

# Step 4: Evaluate the Model (Optional)
train_accuracy = clf.score(X_train, y_train)
test_accuracy = clf.score(X_test, y_test)
print(f"Training Accuracy: {train_accuracy}")
print(f"Testing Accuracy: {test_accuracy}")

# Step 5: Visualize the Decision Tree
plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=iris.feature_names, class_names=iris.target_names)
plt.show()
```

Output:

```
Training Accuracy: 0.9583333333333334
Testing Accuracy: 1.0
```


petal length (cm) <= 2.45
gini = 0.667
samples = 120
value = [40, 41, 39]
class = versicolor
True
False
gini = 0.0
samples = 40
value = [40, 0, 0]
class = setosa
petal length (cm) <= 4.75
gini = 0.5
samples = 80
value = [0, 41, 39]
class = versicolor
petal width (cm) <= 1.65
gini = 0.053
samples = 37
value = [0, 36, 1]
class = versicolor
petal width (cm) <= 1.75
gini = 0.206
samples = 43
value = [0, 5, 38]
class = virginica
gini = 0.0
samples = 36
value = [0, 36, 0]
class = versicolor
gini = 0.0
samples = 1
value = [0, 0, 1]
class = virginica
gini = 0.5
samples = 8
value = [0, 4, 4]
class = versicolor
gini = 0.056
samples = 35
value = [0, 1, 34]
class = virginica

(D). Implement a Support Vector Machine for any relevant dataset.

Code:

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_matrix
from sklearn.datasets import load_iris

# Load the dataset (Iris dataset)
iris = load_iris()
X = iris.data
y = iris.target

# Binarize the target labels for binary classification
# Here we will focus on 'setosa' (class 0) vs. 'non-setosa' (classes 1 and 2)
y_binary = (y == 0).astype(int)  # '1' for Setosa, '0' for non-Setosa

# Split the data into training and testing sets (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y_binary, test_size=0.2, random_state=42)

# Initialize Support Vector Machine model with a linear kernel
svm_model = SVC(kernel='linear')  # You can change kernel to 'rbf' for non-linear

# Train the model on the training set
svm_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = svm_model.predict(X_test)

# Calculate accuracy, precision, and recall
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

# Plot the decision boundary (only for 2D features)
# To simplify, we select two features: 'sepal length' and 'sepal width' for visualization
X_train_2d = X_train[:, :2]  # First two features (sepal length and sepal width)
```

```
X_test_2d = X_test[:, :2]

# Train the model again on the 2D data
svm_model.fit(X_train_2d, y_train)

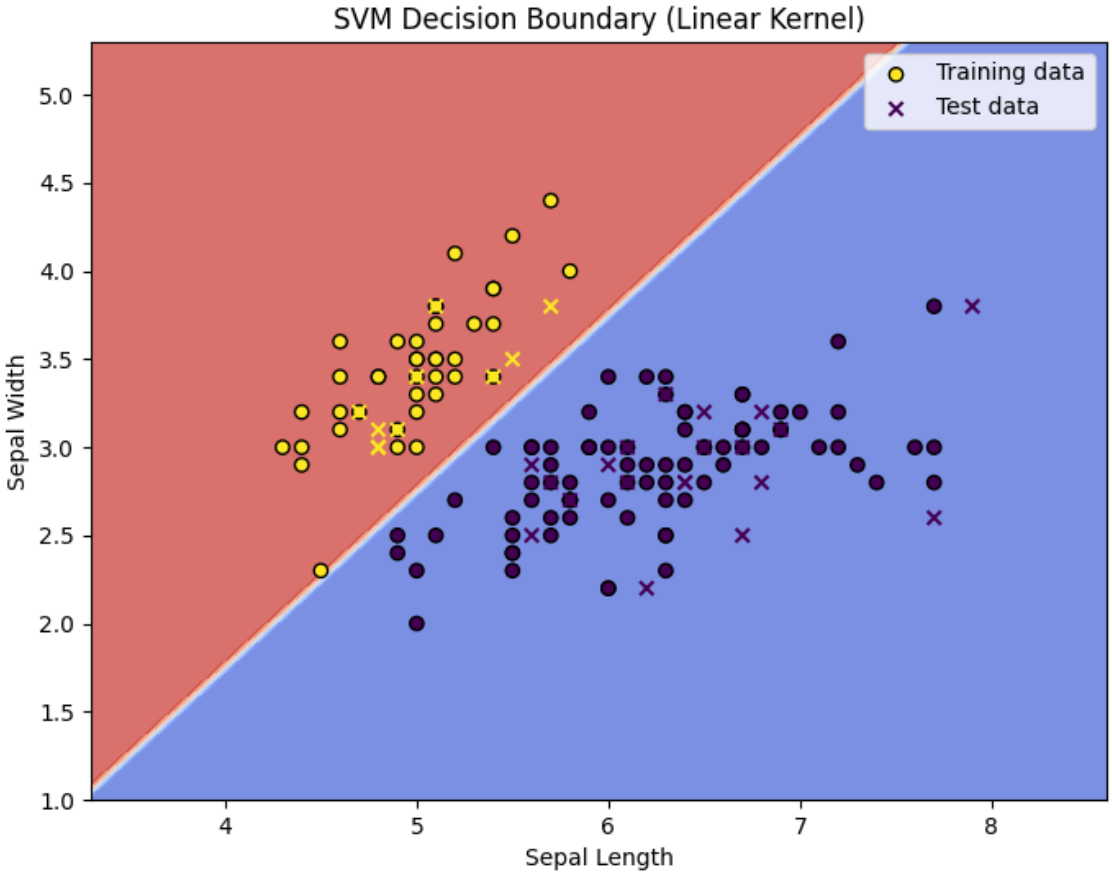
# Create a meshgrid for plotting the decision boundary
x_min, x_max = X_train_2d[:, 0].min() - 1, X_train_2d[:, 0].max() + 1
y_min, y_max = X_train_2d[:, 1].min() - 1, X_train_2d[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                     np.arange(y_min, y_max, 0.1))

# Predict for each point in the meshgrid
Z = svm_model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Plot the decision boundary
plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.75, cmap='coolwarm')
plt.scatter(X_train_2d[:, 0], X_train_2d[:, 1], c=y_train, edgecolors='k', marker='o', label='Training
data')
plt.scatter(X_test_2d[:, 0], X_test_2d[:, 1], c=y_test, edgecolors='r', marker='x', label='Test data')
plt.title('SVM Decision Boundary (Linear Kernel)')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.legend()
plt.show()
```

```
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000

Confusion Matrix:
[[20  0]
 [ 0 10]]
```


SVM Decision Boundary (Linear Kernel)
Training data
Test data
Sepal Width
Sepal Length
5.0
4.5
4.0
3.5
3.0
2.5
2.0
1.5
1.0
4
5
6
7
8

Practical 4(e)

Code:

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.datasets import load_iris

# Load the dataset (Iris dataset)
iris = load_iris()
X = iris.data
y = iris.target

# Split the data into training and testing sets (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 1. Train a single Decision Tree model
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

# Make predictions with the Decision Tree
y_pred_dt = dt_model.predict(X_test)

# Calculate evaluation metrics for the Decision Tree
accuracy_dt = accuracy_score(y_test, y_pred_dt)
precision_dt = precision_score(y_test, y_pred_dt, average='macro')
recall_dt = recall_score(y_test, y_pred_dt, average='macro')

# Print performance for Decision Tree
print("Decision Tree Performance:")
print(f"Accuracy: {accuracy_dt:.4f}")
print(f"Precision: {precision_dt:.4f}")
print(f"Recall: {recall_dt:.4f}\n")

# 2. Train Random Forest models with different numbers of trees (n_estimators) and feature sampling
n_estimators_list = [10, 50, 100, 200]
max_features_list = ['sqrt', 'log2', None]  # Corrected feature sampling strategies
results = []

for n_estimators in n_estimators_list:
    for max_features in max_features_list:
        rf_model = RandomForestClassifier(n_estimators=n_estimators, max_features=max_features,
random_state=42)
        rf_model.fit(X_train, y_train)

        # Make predictions with the Random Forest
        y_pred_rf = rf_model.predict(X_test)

        # Calculate evaluation metrics for Random Forest
```

```
        accuracy_rf = accuracy_score(y_test, y_pred_rf)
        precision_rf = precision_score(y_test, y_pred_rf, average='macro')
        recall_rf = recall_score(y_test, y_pred_rf, average='macro')

        # Save the results
        results.append({
            'n_estimators': n_estimators,
            'max_features': max_features,
            'accuracy': accuracy_rf,
            'precision': precision_rf,
            'recall': recall_rf
        })

# Convert the results to a DataFrame for easier comparison
results_df = pd.DataFrame(results)

# Print Random Forest results for comparison
print("\nRandom Forest Performance with different hyperparameters:")
print(results_df)

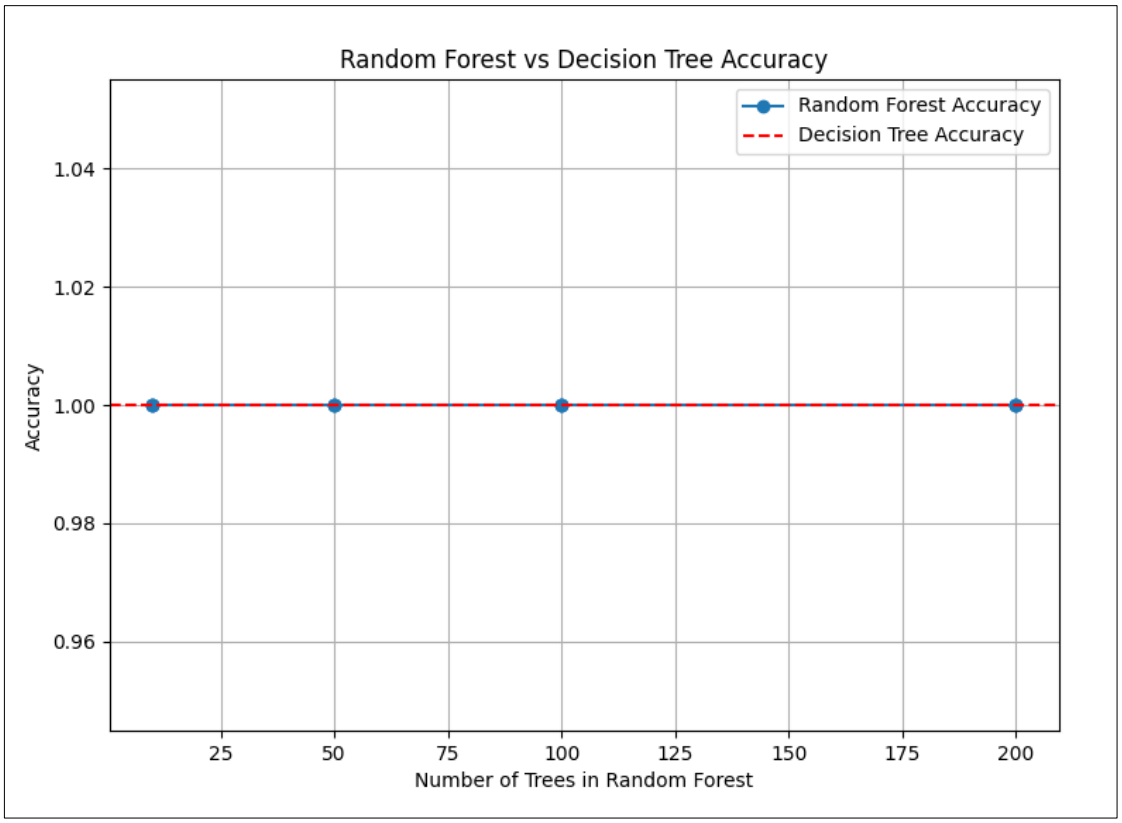
# 3. Compare the performance of Decision Tree and Random Forest models
# We will plot the accuracy scores for both models to visualize the performance
plt.figure(figsize=(10, 6))
plt.plot(results_df['n_estimators'], results_df['accuracy'], label="Random Forest Accuracy",
marker='o')
plt.axhline(y=accuracy_dt, color='r', linestyle='--', label="Decision Tree Accuracy")
plt.title('Random Forest vs Decision Tree Accuracy')
plt.xlabel('Number of Trees in Random Forest')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()

# 4. Plot Precision vs Recall for Random Forest models
plt.figure(figsize=(10, 6))
plt.plot(results_df['n_estimators'], results_df['precision'], label="Random Forest Precision",
marker='o', color='b')
plt.plot(results_df['n_estimators'], results_df['recall'], label="Random Forest Recall", marker='s',
color='g')
plt.axhline(y=precision_dt, color='r', linestyle='--', label="Decision Tree Precision")
plt.axhline(y=recall_dt, color='orange', linestyle='--', label="Decision Tree Recall")
plt.title('Random Forest Precision and Recall vs Decision Tree')
plt.xlabel('Number of Trees in Random Forest')
plt.ylabel('Score')
plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
Decision Tree Performance:
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000


Random Forest Performance with different hyperparameters:
    n_estimators max_features  accuracy  precision  recall
0             10         sqrt       1.0        1.0     1.0
1             10         log2       1.0        1.0     1.0
2             10         None       1.0        1.0     1.0
3             50         sqrt       1.0        1.0     1.0
4             50         log2       1.0        1.0     1.0
5             50         None       1.0        1.0     1.0
6            100         sqrt       1.0        1.0     1.0
7            100         log2       1.0        1.0     1.0
8            100         None       1.0        1.0     1.0
9            200         sqrt       1.0        1.0     1.0
10           200         log2       1.0        1.0     1.0
11           200         None       1.0        1.0     1.0
```



Practical No. 5
Generative Models

a. Implement and demonstrate the working of a Naive Bayesian classifier using a sample data set build the model to classify a test sample Give the practical for this

Implementation:

```
import pandas as pd
import numpy as np
# Step 1: Prepare the data
data = {
   'Age': ['<=30', '<=30', '31-40', '31-40', '>40', '>40', '<=30', '>40', '31-40', '<=30'],
   'Income': ['High', 'Low', 'High', 'Low', 'High', 'Low', 'High', 'Low', 'High', 'Low'],
   'Buys Product': ['No', 'Yes', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'No', 'No', 'Yes']
}
df = pd.DataFrame(data)

# Step 2: Convert categorical data into numeric values
df['Age'] = df['Age'].map({'<=30': 0, '31-40': 1, '>40': 2})
df['Income'] = df['Income'].map({'Low': 0, 'High': 1})
df['Buys Product'] = df['Buys Product'].map({'No': 0, 'Yes': 1})
# Display the dataset
print("Dataset:")
print(df)

# Step 3: Calculate the prior probabilities (P(Buys Product = Yes) and P(Buys Product = No))
prior_yes = df['Buys Product'].sum() / len(df)
prior_no = 1 - prior_yes

# Step 4: Calculate the conditional probabilities (P(Feature|Class))
# For 'Age' and 'Income', we will calculate conditional probabilities for both classes
# For Age given Buys Product = Yes
prob_age_given_yes = df[df['Buys Product'] == 1]['Age'].value_counts(normalize=True).to_dict()
# For Age given Buys Product = No
prob_age_given_no = df[df['Buys Product'] == 0]['Age'].value_counts(normalize=True).to_dict()
# For Income given Buys Product = Yes
prob_income_given_yes = df[df['Buys Product'] ==
1]['Income'].value_counts(normalize=True).to_dict()
# For Income given Buys Product = No
prob_income_given_no = df[df['Buys Product'] ==
0]['Income'].value_counts(normalize=True).to_dict()

# Step 5: Define the Naive Bayes Classifier function
def naive_bayes_predict(age, income):
   # P(Buys Product = Yes)
   prob_yes_given_features = prior_yes
   if age in prob_age_given_yes:
      prob_yes_given_features *= prob_age_given_yes[age]
   if income in prob_income_given_yes:
      prob_yes_given_features *= prob_income_given_yes[income]
    # P(Buys Product = No)
   prob_no_given_features = prior_no
   if age in prob_age_given_no:
```

```
        prob_no_given_features *= prob_age_given_no[age]
    if income in prob_income_given_no:
        prob_no_given_features *= prob_income_given_no[income]
      # Normalize and compare
    total_prob = prob_yes_given_features + prob_no_given_features
    prob_yes_given_features /= total_prob
    prob_no_given_features /= total_prob
        return "Yes" if prob_yes_given_features > prob_no_given_features else "No"

# Step 6: Test the model with a new sample
test_sample = {'Age': 1, 'Income': 1}  # Age = 31-40, Income = High
prediction = naive_bayes_predict(test_sample['Age'], test_sample['Income'])
print("\nPredicted class for the test sample (Age = 31-40, Income = High):", prediction)
```

Output:

Predicted class for the test sample (Age = 31-40, Income = High): No

```
Dataset:
   Age  Income  Buys Product
0    0       1             0
1    0       0             1
2    1       1             1
3    1       0             0
4    2       1             1
5    2       0             1
6    0       1             1
7    2       0             0
8    1       1             0
9    0       0             1

Predicted class for the test sample (Age = 31-40, Income = High): No
```

(B). Implement hidden markov models using hmmlearn give practical for this

Installation:
pip install hmmlearn

Implementation:

```
import numpy as np
from hmmlearn import hmm
# Step 1: Prepare the data
# Observations: 0 = No Umbrella, 1 = Umbrella
observations = np.array([[0], [0], [1], [1], [0], [1], [0], [1]])
# States: 0 = Sunny, 1 = Rainy
# The hidden state sequence is not provided, we will train the HMM model to predict it
# We will only use the observation sequence for training.

# Step 2: Define and Train the HMM Model
# Define the HMM: 2 hidden states (Sunny, Rainy), 2 possible observations (No Umbrella, Umbrella)
model = hmm.MultinomialHMM(n_components=2, n_iter=1000)
# Train the model on the observations (without the hidden state labels)
model.fit(observations)

# Step 3: Predict the hidden states (weather) given the observations
predicted_states = model.predict(observations)

# Step 4: Output results
print("Predicted Hidden States (Weather):")
print(predicted_states)

# Step 5: Output the model's parameters (transition matrix, emission matrix)
print("\nTransition Matrix (State to State):")
print(model.transmat_)
print("\nEmission Matrix (Observation | State):")
print(model.emissionprob_)

# Step 6: Making predictions for new observations (example)
# Let's predict the weather based on a new sequence of umbrella usage
new_observations = np.array([[0], [1], [1]])  # No Umbrella, Umbrella, Umbrella
predicted_new_states = model.predict(new_observations)
print("\nPredicted Hidden States for New Observations (No Umbrella, Umbrella, Umbrella):")
print(predicted_new_states)
```

Output:

```
Predicted Hidden States (Weather):
[1 0 1 0 1 0 1 0]

Transition Matrix (State to State):
[[0.16300117 0.83699883]
 [0.91569981 0.08430019]]

Emission Matrix (Observation | State):
[[1.]
 [1.]]

Predicted Hidden States for New Observations (No Umbrella, Umbrella, Umbrella):
[1 0 1]
```

Practical No. 6
Probabilistic Models

(A). Implement Bayesian linear regression to explore prior and posterior distribution.

Implementation:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import multivariate_normal
# Set random seed for reproducibility
np.random.seed(42)

# 1. Generate synthetic data (linear relationship with noise)
n_samples = 50
X = np.linspace(0, 10, n_samples)
y_true = 2 * X + 1  # y = 2x + 1 (true coefficients)
noise = np.random.normal(0, 1, n_samples)
y = y_true + noise  # observed data with noise
# Plot the data
plt.figure(figsize=(8, 6))
plt.scatter(X, y, label="Observed data", color='blue')
plt.plot(X, y_true, label="True line", color='red', linewidth=2)
plt.xlabel('X')
plt.ylabel('y')
plt.title('Synthetic Data and True Line')
plt.legend()
plt.show()

# 2. Define prior distributions for the parameters (weights)
# Assuming a Normal prior distribution for the weights (slope and intercept)
# We assume a zero-mean prior with a high variance (uninformative prior)
prior_mean = np.array([0, 0])  # [slope, intercept]
prior_cov = np.array([[10, 0], [0, 10]])  # Uncorrelated, high variance

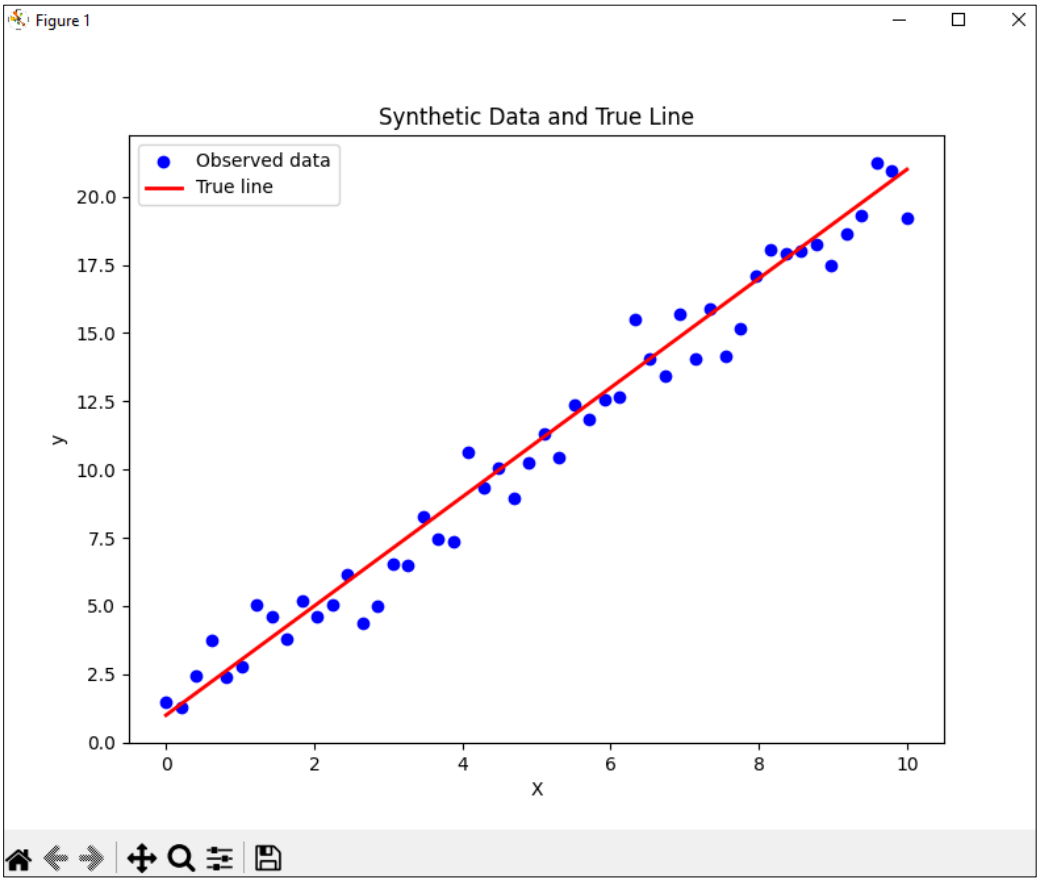
# 3. Compute the posterior distribution using Bayesian formula
# We use the equation: p(w | X, y) ~ p(y | X, w) * p(w)
# p(y | X, w) is the likelihood: N(y | Xw, sigma^2)
# p(w) is the prior: N(w | prior_mean, prior_cov)
# Design matrix (with a column of ones for the intercept)
X_design = np.vstack([X, np.ones_like(X)]).T
# Likelihood covariance (assuming noise variance is known)
sigma_squared = 1  # Variance of the Gaussian noise
likelihood_cov = sigma_squared * np.identity(n_samples)
# Posterior mean and covariance (using the closed-form solution for Bayesian LR)
X_transpose = X_design.T
posterior_cov = np.linalg.inv(np.linalg.inv(prior_cov) + X_transpose @ np.linalg.inv(likelihood_cov)
@ X_design)
posterior_mean = posterior_cov @ (np.linalg.inv(prior_cov) @ prior_mean + X_transpose @
np.linalg.inv(likelihood_cov) @ y)

# 4. Plot the prior and posterior distributions
# We will visualize the prior and posterior distributions of the slope and intercept
```

```
sns.set(style="whitegrid")

# Plot prior distribution
plt.figure(figsize=(10, 6))
plt.subplot(1, 2, 1)
slope_range = np.linspace(-5, 5, 100)
intercept_range = np.linspace(-5, 5, 100)
slope_grid, intercept_grid = np.meshgrid(slope_range, intercept_range)
prior_pdf = multivariate_normal.pdf(
    np.dstack([slope_grid, intercept_grid]), mean=prior_mean, cov=prior_cov)
plt.contour(slope_grid, intercept_grid, prior_pdf, levels=10, cmap='Blues')
plt.title('Prior Distribution')
plt.xlabel('Slope')
plt.ylabel('Intercept')
# Plot posterior distribution
plt.subplot(1, 2, 2)
posterior_pdf = multivariate_normal.pdf(
    np.dstack([slope_grid, intercept_grid]), mean=posterior_mean, cov=posterior_cov)
plt.contour(slope_grid, intercept_grid, posterior_pdf, levels=10, cmap='Reds')
plt.title('Posterior Distribution')
plt.xlabel('Slope')
plt.ylabel('Intercept')
plt.tight_layout()
plt.show()
# 5. Make predictions based on the posterior distribution
# Draw samples from the posterior distribution to make predictions
n_samples_posterior = 500
posterior_samples = np.random.multivariate_normal(posterior_mean, posterior_cov,
n_samples_posterior)
# Generate predictions using the posterior samples
predictions = np.array([X_design @ sample for sample in posterior_samples])
# Plot the data and the predictive distribution
plt.figure(figsize=(8, 6))
plt.scatter(X, y, color='blue', label='Observed data')
plt.plot(X, y_true, label='True Line', color='red', linewidth=2)
plt.plot(X, np.mean(predictions, axis=0), label='Posterior mean prediction', color='green',
linewidth=2)
# Plot the uncertainty in predictions (shaded area)
std_dev = np.std(predictions, axis=0)
plt.fill_between(X, np.mean(predictions, axis=0) - 1.96 * std_dev, np.mean(predictions, axis=0) +
1.96 * std_dev, color='green', alpha=0.2, label='95% prediction interval')
plt.xlabel('X')
plt.ylabel('y')
plt.title('Posterior Predictions with Uncertainty')
plt.legend()
plt.show()
```

Output:


Figure 1
Synthetic Data and True Line
Observed data
True line
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
y
0
2
4
6
8
10
X

(B). Implement gaussian mixture models for density estimation and unsupervised clustering

Installation:
pip install numpy matplotlib scikit-learn

Implementation:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture
from sklearn.datasets import make_blobs
# Step 1: Generate Synthetic Data (Multiple Gaussian Distributions)
np.random.seed(42)

# Create synthetic data: 3 Gaussian distributions
n_samples = 500
X, _ = make_blobs(n_samples=n_samples, centers=3, cluster_std=0.60, random_state=42)

# Step 2: Fit a Gaussian Mixture Model (GMM) to the data
# Fit a GMM with 3 components (since we know there are 3 clusters)
gmm = GaussianMixture(n_components=3)
gmm.fit(X)

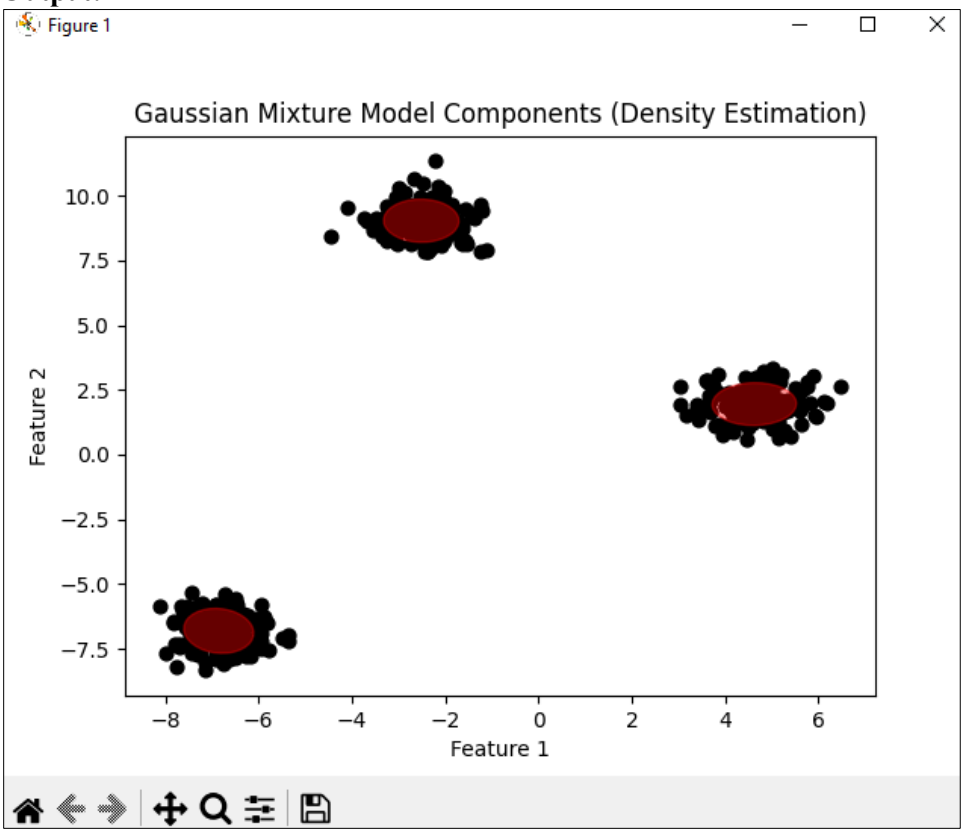
# Step 3: Visualize the data points and the GMM
# Plot the data points
plt.scatter(X[:, 0], X[:, 1], c='black', s=40, label="Data")
# Plot the GMM components (mean of each Gaussian)
means = gmm.means_
covariances = gmm.covariances_
# Plot the ellipses for each Gaussian component (representing the covariance)
for mean, cov in zip(means, covariances):
    v, w = np.linalg.eigh(cov)
    v = 2.0 * np.sqrt(2.0) * np.sqrt(v)
    u = w[0] / np.linalg.norm(w[0])
    angle = np.arctan(u[1] / u[0])
    angle = 180.0 * angle / np.pi
    ell = plt.matplotlib.patches.Ellipse(mean, v[0], v[1], angle=angle, color='red', alpha=0.4)
    plt.gca().add_patch(ell)
plt.title('Gaussian Mixture Model Components (Density Estimation)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()

# Step 4: Use the GMM for clustering (predict the cluster for each point)
labels = gmm.predict(X)

# Step 5: Visualize the clustering result
plt.scatter(X[:, 0], X[:, 1], c=labels, s=40, cmap='viridis', label="Clustered Data")
plt.title('Clustering Results from GMM')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

```
# Step 6: Density estimation - Plot the GMM's predicted density on a grid
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 100))
grid_data = np.column_stack([xx.ravel(), yy.ravel()])
# Get the log likelihood (density) of the data points under the GMM
Z = np.exp(gmm.score_samples(grid_data))
Z = Z.reshape(xx.shape)
# Plot the density
plt.contourf(xx, yy, Z, levels=10, cmap='Blues')
plt.scatter(X[:, 0], X[:, 1], c='black', s=40, alpha=0.5, label="Data")
plt.title('Density Estimation with GMM')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Output:



Practical No. 7
Model Evaluation and Hyperparameter Tuning

(A). Implement cross-validation techniques (k-fold, stratified, etc.) for robust model evaluation

Implementation

```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score, KFold, StratifiedKFold
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import make_scorer, accuracy_score, f1_score
# Load dataset
data = load_iris()
X, y = data.data, data.target

# Initialize model
model = DecisionTreeClassifier(random_state=42)

# Define k-Fold Cross-Validation
k = 5  # Number of folds
kfold = KFold(n_splits=k, shuffle=True, random_state=42)

# Evaluate with k-Fold CV
kfold_scores = cross_val_score(model, X, y, cv=kfold, scoring='accuracy')
print(f"k-Fold Cross-Validation Accuracy: {kfold_scores}")
print(f"Mean Accuracy (k-Fold): {kfold_scores.mean():.2f}")

# Define Stratified k-Fold Cross-Validation
stratified_kfold = StratifiedKFold(n_splits=k, shuffle=True, random_state=42)

# Evaluate with Stratified k-Fold CV
stratified_scores = cross_val_score(model, X, y, cv=stratified_kfold, scoring='accuracy')
print(f"Stratified k-Fold Cross-Validation Accuracy: {stratified_scores}")
print(f"Mean Accuracy (Stratified k-Fold): {stratified_scores.mean():.2f}")

# Advanced: Custom Scorer with F1-Score
f1_scorer = make_scorer(f1_score, average='weighted')
f1_scores = cross_val_score(model, X, y, cv=stratified_kfold, scoring=f1_scorer)
print(f"Stratified k-Fold Cross-Validation F1-Score: {f1_scores}")
print(f"Mean F1-Score (Stratified k-Fold): {f1_scores.mean():.2f}")
```

Output :

```
k-Fold Cross-Validation Accuracy: [1.         0.96666667 0.93333333 0.93333333 0.93333333]
Mean Accuracy (k-Fold): 0.95
Stratified k-Fold Cross-Validation Accuracy: [1.         0.96666667 0.93333333 0.96666667
0.9       ]
Mean Accuracy (Stratified k-Fold): 0.95
Stratified k-Fold Cross-Validation F1-Score: [1.         0.96658312 0.93265993 0.96658312
0.89974937]
Mean F1-Score (Stratified k-Fold): 0.95
Mean Accuracy (LOOCV): 0.94
Mean Accuracy (Time-Series Split): 0.58
```

(B). Systematically explore combinations of hyperparameters to optimize model performance. (use grid and randomized search)

Implementation

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# Load dataset
data = load_iris()
X, y = data.data, data.target

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Define the model
model = RandomForestClassifier(random_state=42)

# Define the hyperparameter grid for Grid Search
param_grid = {
    'n_estimators': [10, 50, 100, 200],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
# Perform Grid Search
grid_search = GridSearchCV(estimator=model, param_grid=param_grid, cv=5, scoring='accuracy',
verbose=1)
grid_search.fit(X_train, y_train)
# Print best parameters and accuracy
print("Best Parameters (Grid Search):", grid_search.best_params_)
print("Best Accuracy (Grid Search):", grid_search.best_score_)
# Define the hyperparameter grid for Randomized Search
param_dist = {
    'n_estimators': [int(x) for x in np.linspace(10, 200, num=20)],
    'max_depth': [None] + [int(x) for x in np.linspace(10, 30, num=5)],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'bootstrap': [True, False]
}

# Perform Randomized Search
random_search = RandomizedSearchCV(estimator=model, param_distributions=param_dist,
n_iter=50,
                                   cv=5, scoring='accuracy', random_state=42, verbose=1)
random_search.fit(X_train, y_train)
# Print best parameters and accuracy
print("Best Parameters (Randomized Search):", random_search.best_params_)
print("Best Accuracy (Randomized Search):", random_search.best_score_)
# Evaluate the best model on the test set
best_model = random_search.best_estimator_
```

```
y_pred = best_model.predict(X_test)
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Output

```
Fitting 5 folds for each of 144 candidates, totalling 720 fits
Best Parameters (Grid Search): {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2,
'n_estimators': 100}
Best Accuracy (Grid Search): 0.9428571428571428
Fitting 5 folds for each of 50 candidates, totalling 250 fits
Best Parameters (Randomized Search): {'n_estimators': 90, 'min_samples_split': 2, 'min_samples_lea
f': 2, 'max_depth': 25, 'bootstrap': True}
Best Accuracy (Randomized Search): 0.9428571428571428

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

Practical No. 8
Bayesian Learning

(A). Implement Bayesian learning using inferences

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import multivariate_normal
# Set random seed for reproducibility
np.random.seed(42)
# 1. Generate synthetic data (linear relationship with noise)
n_samples = 50
X = np.linspace(0, 10, n_samples)
y_true = 2 * X + 1  # y = 2x + 1 (true coefficients)
noise = np.random.normal(0, 1, n_samples)
y = y_true + noise  # observed data with noise

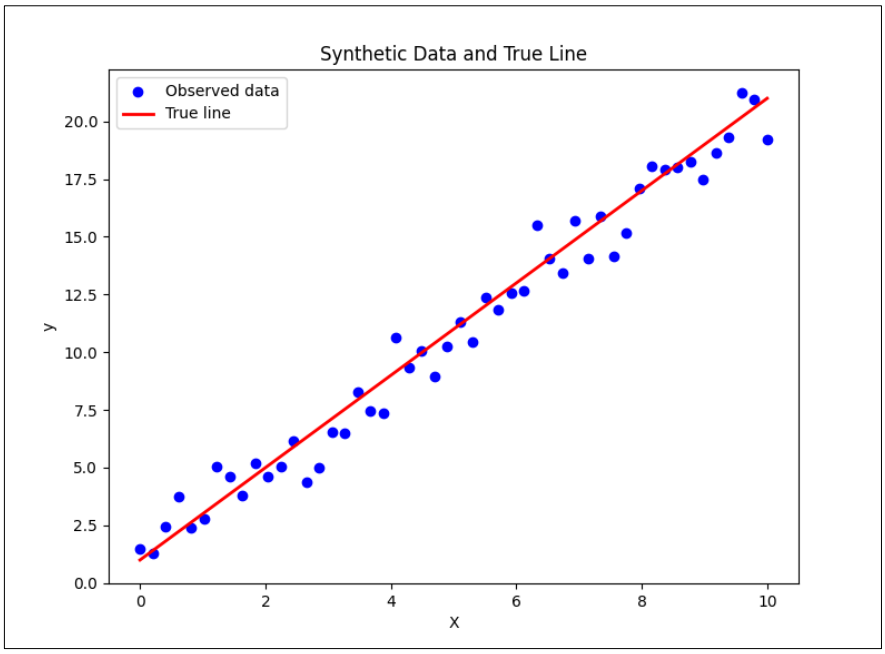
# Plot the data
plt.figure(figsize=(8, 6))
plt.scatter(X, y, label="Observed data", color='blue')
plt.plot(X, y_true, label="True line", color='red', linewidth=2)
plt.xlabel('X')
plt.ylabel('y')
plt.title('Synthetic Data and True Line')
plt.legend()
plt.show()
# 2. Define prior distributions for the parameters (weights)
# Assuming a Normal prior distribution for the weights (slope and intercept)
# We assume a zero-mean prior with a high variance (uninformative prior)
prior_mean = np.array([0, 0])  # [slope, intercept]
prior_cov = np.array([[10, 0], [0, 10]])  # Uncorrelated, high variance
# 3. Compute the posterior distribution using Bayesian formula
# We use the equation: p(w | X, y) ~ p(y | X, w) * p(w)
# p(y | X, w) is the likelihood: N(y | Xw, sigma^2)
# p(w) is the prior: N(w | prior_mean, prior_cov)
# Design matrix (with a column of ones for the intercept)
X_design = np.vstack([X, np.ones_like(X)]).T
# Likelihood covariance (assuming noise variance is known)
sigma_squared = 1  # Variance of the Gaussian noise
likelihood_cov = sigma_squared * np.identity(n_samples)
# Posterior mean and covariance (using the closed-form solution for Bayesian LR)
X_transpose = X_design.T
posterior_cov = np.linalg.inv(np.linalg.inv(prior_cov) + X_transpose @ np.linalg.inv(likelihood_cov)
@ X_design)
posterior_mean = posterior_cov @ (np.linalg.inv(prior_cov) @ prior_mean + X_transpose @
np.linalg.inv(likelihood_cov) @ y)
# 4. Plot the prior and posterior distributions
# We will visualize the prior and posterior distributions of the slope and intercept
sns.set(style="whitegrid")
# Plot prior distribution
```

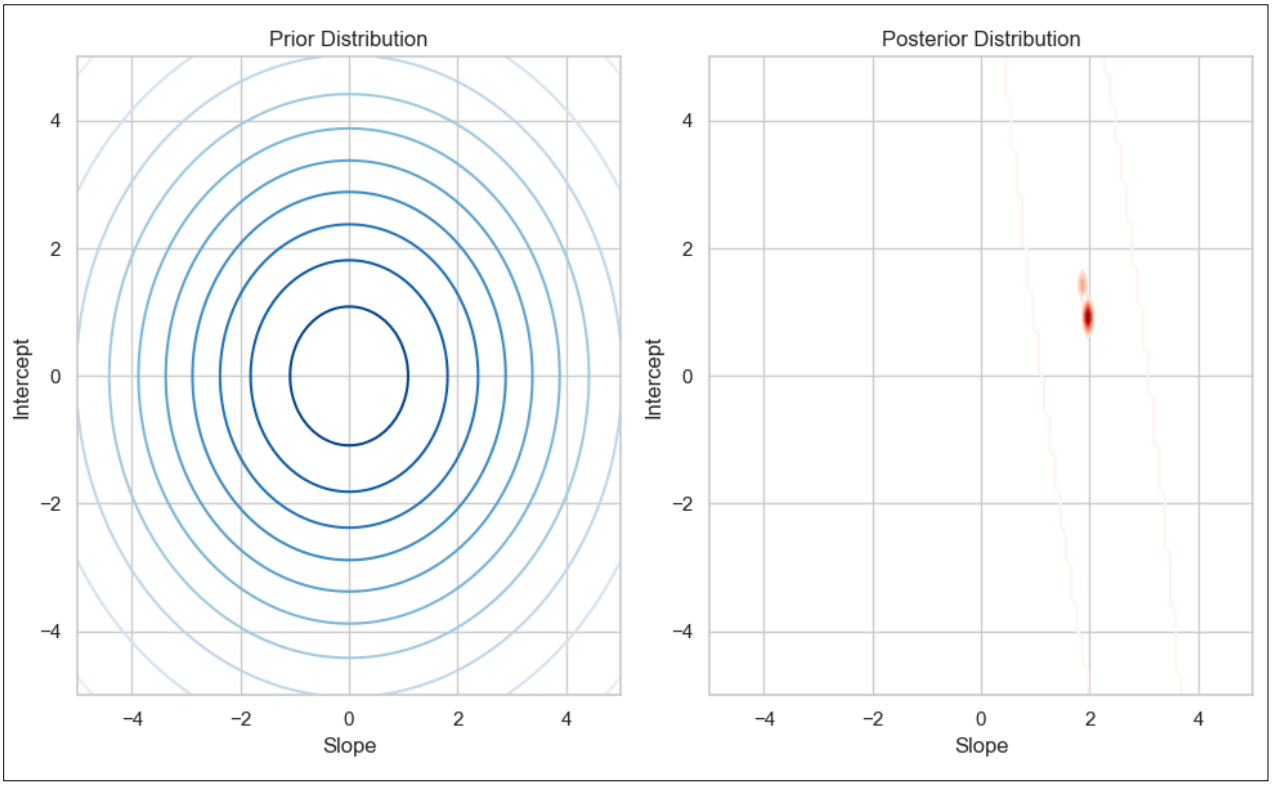
```
plt.figure(figsize=(10, 6))
plt.subplot(1, 2, 1)
slope_range = np.linspace(-5, 5, 100)
intercept_range = np.linspace(-5, 5, 100)
slope_grid, intercept_grid = np.meshgrid(slope_range, intercept_range)
prior_pdf = multivariate_normal.pdf(
    np.dstack([slope_grid, intercept_grid]), mean=prior_mean, cov=prior_cov)

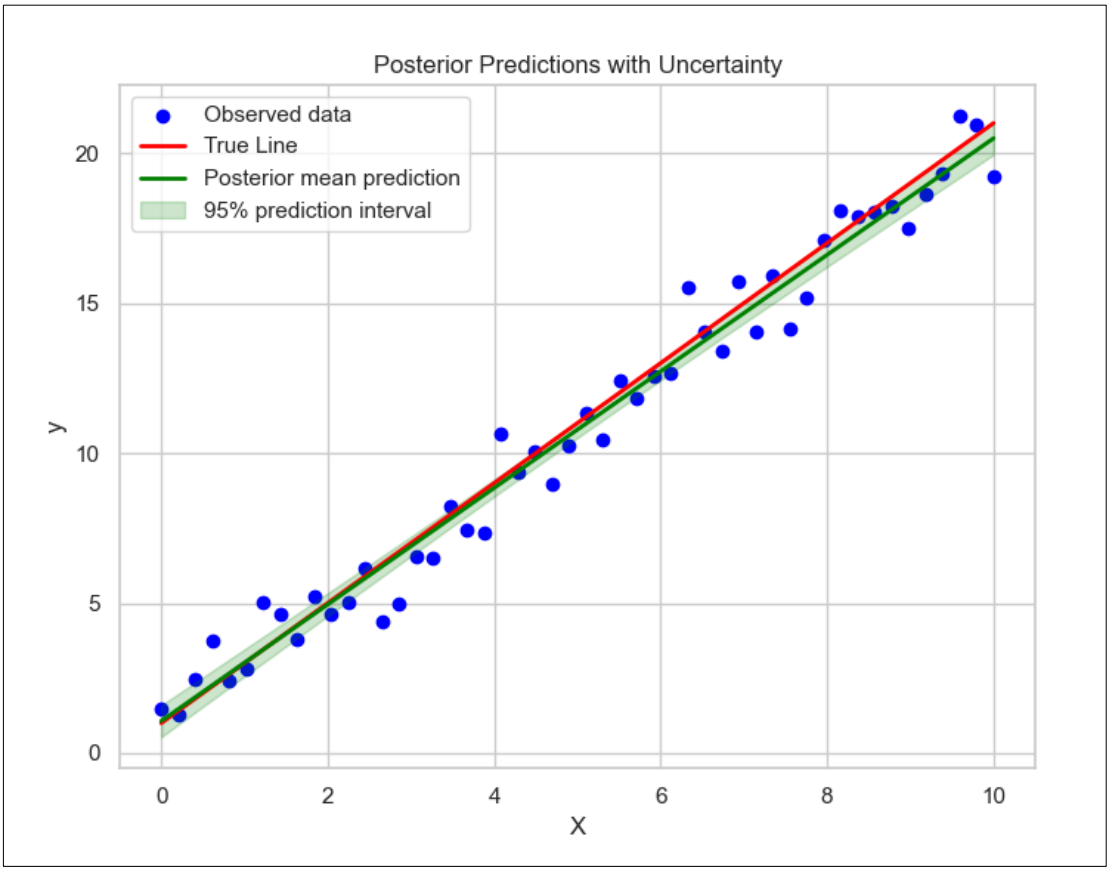
plt.contour(slope_grid, intercept_grid, prior_pdf, levels=10, cmap='Blues')
plt.title('Prior Distribution')
plt.xlabel('Slope')
plt.ylabel('Intercept')

# Plot posterior distribution
plt.subplot(1, 2, 2)
posterior_pdf = multivariate_normal.pdf(
    np.dstack([slope_grid, intercept_grid]), mean=posterior_mean, cov=posterior_cov)
plt.contour(slope_grid, intercept_grid, posterior_pdf, levels=10, cmap='Reds')
plt.title('Posterior Distribution')
plt.xlabel('Slope')
plt.ylabel('Intercept')
plt.tight_layout()
plt.show()
# 5. Make predictions based on the posterior distribution
# Draw samples from the posterior distribution to make predictions
n_samples_posterior = 500
posterior_samples = np.random.multivariate_normal(posterior_mean, posterior_cov,
n_samples_posterior)
# Generate predictions using the posterior samples
predictions = np.array([X_design @ sample for sample in posterior_samples])
# Plot the data and the predictive distribution
plt.figure(figsize=(8, 6))
plt.scatter(X, y, color='blue', label='Observed data')
plt.plot(X, y_true, label='True Line', color='red', linewidth=2)
plt.plot(X, np.mean(predictions, axis=0), label='Posterior mean prediction', color='green',
linewidth=2)
# Plot the uncertainty in predictions (shaded area)
std_dev = np.std(predictions, axis=0)
plt.fill_between(X, np.mean(predictions, axis=0) - 1.96 * std_dev, np.mean(predictions, axis=0) +
1.96 * std_dev, color='green', alpha=0.2, label='95% prediction interval')
plt.xlabel('X')
plt.ylabel('y')
plt.title('Posterior Predictions with Uncertainty')
plt.legend()
plt.show()
```

Output:


Synthetic Data and True Line
Observed data
True line
y
X
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
0
2
4
6
8
10


Prior Distribution
Posterior Distribution
Intercept
Slope
4
2
0
−2
−4


Posterior Predictions with Uncertainty
Observed data
True Line
Posterior mean prediction
95% prediction interval
y
X
0
5
10
15
20
0
2
4
6
8
10

Practical No. 9
Deep Generative Models

(A). Set up a generator network to produce samples and a discriminator network to distinguish between real and generated data. (Use a simple small dataset)

Code:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt

# Step 1: Create a simple dataset (1D Gaussian distribution)
def generate_real_data(n_samples):
    X = np.random.normal(loc=0.0, scale=1.0, size=n_samples)  # Mean=0, Stddev=1
    y = np.ones((n_samples, 1))  # Labels for real data
    return X, y

# Step 2: Define the Generator
def build_generator(latent_dim):
    model = Sequential([
        Dense(16, activation='relu', input_dim=latent_dim),
        Dense(1, activation='linear')  # Output is a single number (1D point)
    ])
    return model
# Step 3: Define the Discriminator
def build_discriminator():
    model = Sequential([
        Dense(16, activation='relu', input_dim=1),
        Dense(1, activation='sigmoid')  # Output is a probability
    ])
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    return model

# Step 4: Define the GAN
def build_gan(generator, discriminator):
    discriminator.trainable = False  # Freeze the discriminator during GAN training
    model = Sequential([generator, discriminator])
    model.compile(optimizer='adam', loss='binary_crossentropy')
    return model

# Step 5: Train the GAN
def train_gan(generator, discriminator, gan, latent_dim, n_epochs=10000, n_batch=128,
n_eval=1000):
    for epoch in range(n_epochs):
        # Train the Discriminator
```

```
        # 1. Real data
        X_real, y_real = generate_real_data(n_batch // 2)
        d_loss_real = discriminator.train_on_batch(X_real, y_real)

        # 2. Fake data
        X_fake = generator.predict(np.random.randn(n_batch // 2, latent_dim))
        y_fake = np.zeros((n_batch // 2, 1))  # Labels for fake data
        d_loss_fake = discriminator.train_on_batch(X_fake, y_fake)

        # Train the Generator (via the GAN model)
        z_input = np.random.randn(n_batch, latent_dim)  # Random noise
        y_gan = np.ones((n_batch, 1))  # Labels for GAN training are always 1
        g_loss = gan.train_on_batch(z_input, y_gan)

        # Evaluate progress
        if (epoch + 1) % n_eval == 0:
            print(f"Epoch {epoch+1}/{n_epochs}, D Loss Real: {d_loss_real[0]:.3f}, D Loss
Fake: {d_loss_fake[0]:.3f}, G Loss: {g_loss:.3f}")
            summarize_performance(epoch, generator, latent_dim)

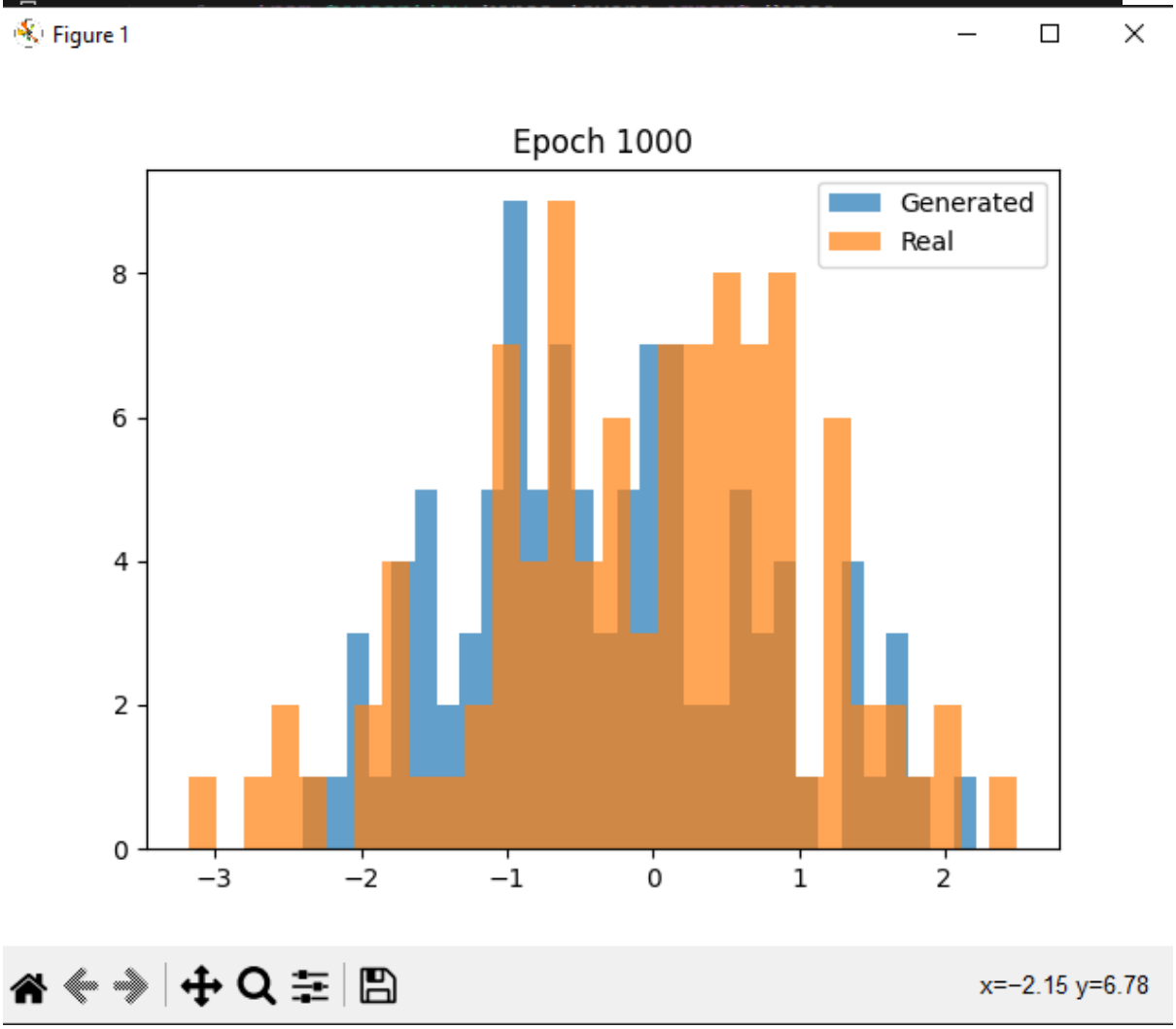
# Step 6: Evaluate and Plot Results
def summarize_performance(epoch, generator, latent_dim, n_samples=100):
    X_fake = generator.predict(np.random.randn(n_samples, latent_dim))
    plt.hist(X_fake, bins=30, alpha=0.7, label='Generated')
    plt.hist(np.random.normal(0.0, 1.0, n_samples), bins=30, alpha=0.7, label='Real')
    plt.title(f'Epoch {epoch+1}')
    plt.legend()
    plt.show()

# Parameters
latent_dim = 5  # Size of random noise vector
n_epochs = 1000
n_batch = 32

# Build and compile models
generator = build_generator(latent_dim)
discriminator = build_discriminator()
gan = build_gan(generator, discriminator)
# Train the GAN
train_gan(generator, discriminator, gan, latent_dim, n_epochs, n_batch, n_eval=1000)
```

Output:

```
1/1 [==============================] - 0s 46ms/step
Epoch 1000/1000, D Loss Real: 0.666, D Loss Fake: 0.742, G Loss: 0.688
4/4 [==============================] - 0s 2ms/step
```



Practical No. 10

Develop an API to deploy your model and perform predictions

Code:

```
# Import necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score
import joblib
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris

# Load the Iris dataset from sklearn
iris = load_iris()
# Convert the data into a DataFrame for easier handling
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df['target'] = iris.target
# Quick inspection of the dataset
print(df.head())
print(df.info())
# Data Preprocessing
# Features (X) are all columns except 'target'
X = df.drop(columns=['target'])
y = df['target']

# Split the dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Initialize the model (RandomForestClassifier in this case)
model = RandomForestClassifier(n_estimators=100, random_state=42)
# Train the model
model.fit(X_train, y_train)
# Make predictions on the test set
y_pred = model.predict(X_test)

# Evaluate the model
print("Accuracy Score: ", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

# Save the trained model using joblib
joblib.dump(model, 'trained_model.joblib')

# Optionally: Visualize feature importance (if the model allows it)
features = X.columns
importances = model.feature_importances_
indices = np.argsort(importances)

plt.figure(figsize=(10, 6))
```

```
plt.title("Feature Importances")
plt.barh(range(X.shape[1]), importances[indices], align="center")
plt.yticks(range(X.shape[1]), features[indices])
plt.xlabel("Relative Importance")
plt.show()

print("Model saved as 'trained_model.joblib'")
```

Step 2: Load the Model and Create the API Using FastAPI

Once you have the model saved, create an API using FastAPI. This API will accept inputs via POST requests and return predictions.

Create a file called main.py:

```
# main.py
```

Code:

```
import joblib
from fastapi import FastAPI
from pydantic import BaseModel
from sklearn.datasets import load_iris
import pandas as pd

# Load the trained model from the .joblib file
model = joblib.load('trained_model.joblib')
# Define the input data structure using Pydantic
class FlowerInput(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float
# Initialize the FastAPI app
app = FastAPI()

# Define a function to make predictions
def predict_species(sepal_length, sepal_width, petal_length, petal_width):
    # Prepare the input data as a DataFrame
    input_data = pd.DataFrame([[sepal_length, sepal_width, petal_length, petal_width]],
                              columns=['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width
(cm)'])

    # Make predictions
    prediction = model.predict(input_data)

    # Map prediction to actual species names
    iris = load_iris()
    species = iris.target_names[prediction]
    return species[0]

# Define an endpoint for prediction
@app.post("/predict/")
async def predict(flowers: FlowerInput):
    # Call the prediction function
```

```
    predicted_species = predict_species(flowers.sepal_length, flowers.sepal_width,
flowers.petal_length, flowers.petal_width)

    # Return the result as a JSON response
    return {"predicted_species": predicted_species}
```

Step 3: Run the API

To run the API, use the following command:
uvicorn main:app --reload
This will start a FastAPI app on http://127.0.0.1:8000. You can now make predictions by sending POST requests.

Step 4: Test Your API

You can now test your API using a terminal command (in PowerShell or Command Prompt on Windows).
Here's how you can call your FastAPI using PowerShell with Invoke-WebRequest:

```
$headers = @{
    "accept" = "application/json"
    "Content-Type" = "application/json"
}

$data = '{"sepal_length": 9, "sepal_width": 9, "petal_length": 0, "petal_width": 9}'

Invoke-WebRequest -Uri "http://127.0.0.1:8000/predict/" -Method Post -Headers $headers -Body
$data
```

Expected Output:

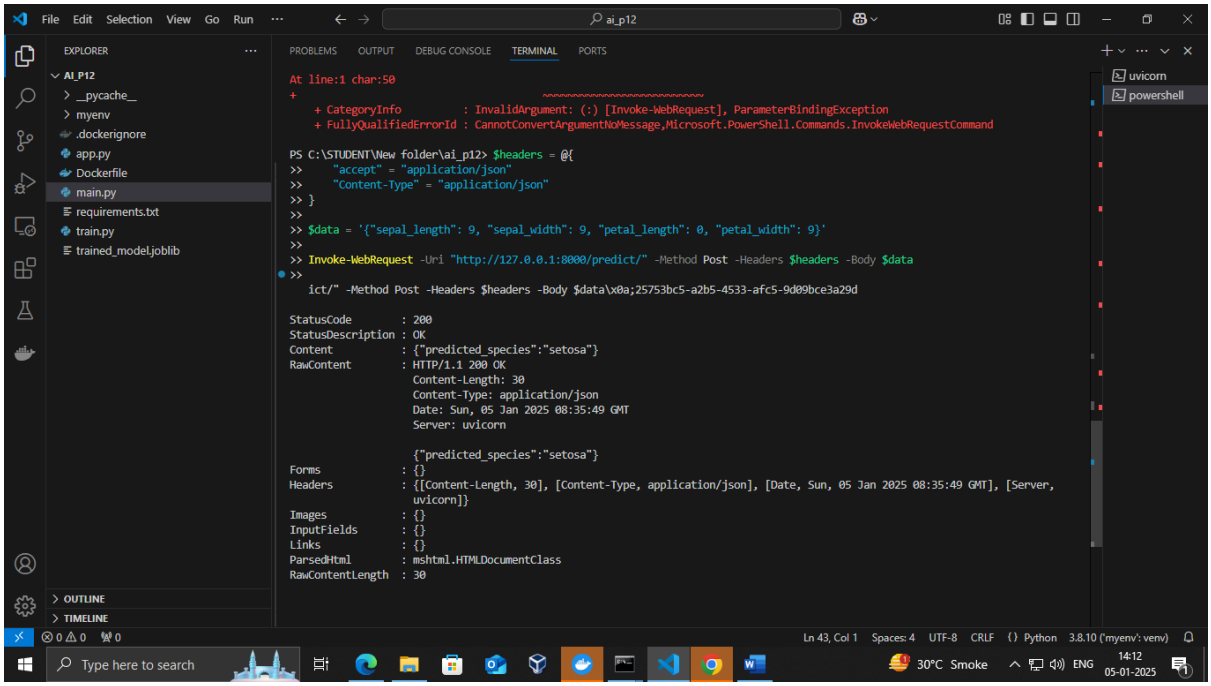
The API will return a response with the predicted species. The response will look like:

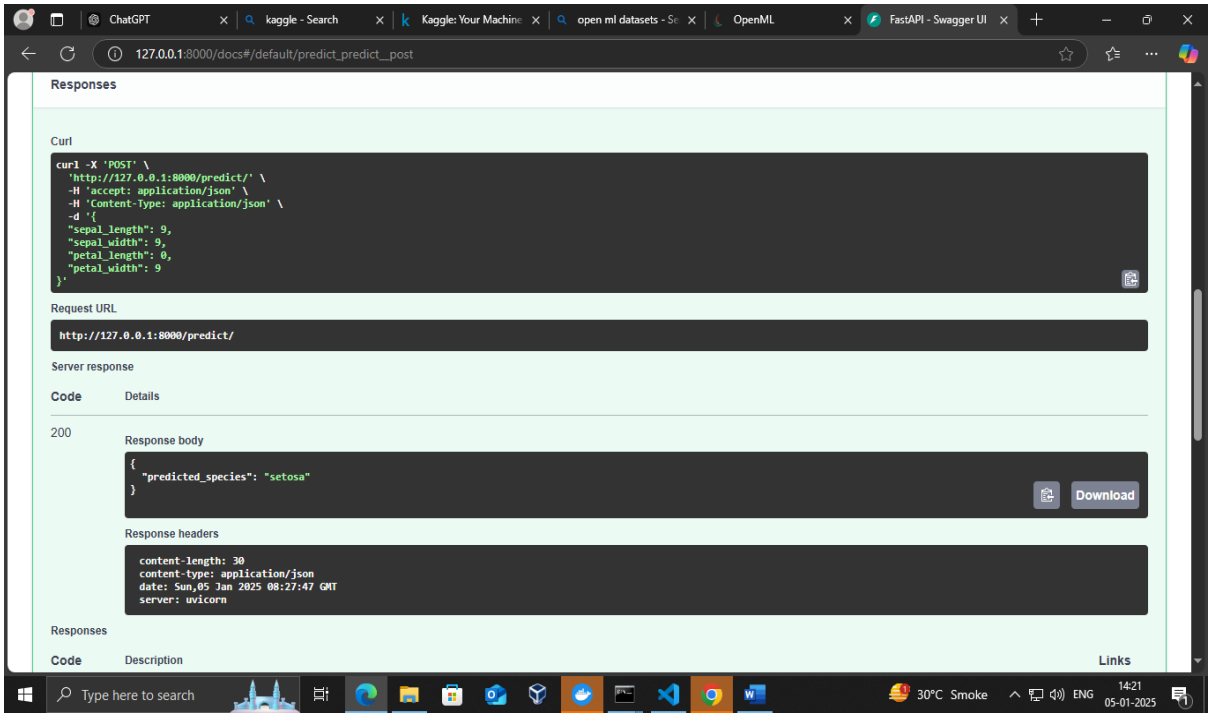
```
{
  "predicted_species": "setosa"
}
```

This confirms that the API is working, and the model is making predictions based on the input features.

First, create a requirements.txt file for your project dependencies.
requirements.txt
Copy code

```
fastapi==0.95.1
uvicorn==0.23.1
scikit-learn==1.2.0
pandas==1.5.3
joblib==1.2.0
numpy==1.23.5
matplotlib==3.6.3
seaborn==0.12.2
```


File Edit Selection View Go Run
ai_p12
EXPLORER
AI_P12
__pycache__
myenv
.dockerignore
app.py
Dockerfile
main.py
requirements.txt
train.py
trained_model.joblib
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
uvicorn
powershell
At line:1 char:50
+ CategoryInfo : InvalidArgument: (:) [Invoke-WebRequest], ParameterBindingException
+ FullyQualifiedErrorId : CannotConvertArgumentNoMessage,Microsoft.PowerShell.Commands.InvokeWebRequestCommand
PS C:\STUDENT\New folder\ai_p12> $headers = @{
"accept" = "application/json"
"Content-Type" = "application/json"
}
$data = '{"sepal_length": 9, "sepal_width": 9, "petal_length": 0, "petal_width": 9}'
Invoke-WebRequest -Uri "http://127.0.0.1:8000/predict/" -Method Post -Headers $headers -Body $data
StatusCode : 200
StatusDescription : OK
Content : {"predicted_species":"setosa"}
RawContent : HTTP/1.1 200 OK
Content-Length: 30
Content-Type: application/json
Date: Sun, 05 Jan 2025 08:35:49 GMT
Server: uvicorn
{"predicted_species":"setosa"}
Forms : {}
Headers : {[Content-Length, 30], [Content-Type, application/json], [Date, Sun, 05 Jan 2025 08:35:49 GMT], [Server, uvicorn]}
Images : {}
InputFields : {}
Links : {}
ParsedHtml : mshtml.HTMLDocumentClass
RawContentLength : 30
OUTLINE
TIMELINE
Ln 43, Col 1 Spaces: 4 UTF-8 CRLF Python 3.8.10 ('myenv': venv)
Type here to search
30°C Smoke
ENG
14:12
05-01-2025


ChatGPT
kaggle - Search
Kaggle: Your Machine
open ml datasets - Se
OpenML
FastAPI - Swagger UI
127.0.0.1:8000/docs#/default/predict_predict__post
Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8000/predict/' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
"sepal_length": 9,
"sepal_width": 9,
"petal_length": 0,
"petal_width": 9
}'
Request URL
http://127.0.0.1:8000/predict/
Server response
Code Details
200
Response body
{
"predicted_species": "setosa"
}
Download
Response headers
content-length: 30
content-type: application/json
date: Sun,05 Jan 2025 08:27:47 GMT
server: uvicorn
Responses
Code Description Links
Type here to search
30°C Smoke
ENG
14:21
05-01-2025